Machine Programming

Lecture 18 – PL Kaleidoscope IV: Software Testing and Dynamic Analysis

Ziyang Li

Logistics

- Oral presentation
  - The papers will be sent out today via email!
  - Attendence will be recorded unless the course staff are notified!
- Final project proposal
  - Final project proposal due this Sunday!

Requirements

- **Format:** Write it in **Markdown** and submit the `.md` file.
- **Length: At most one page**, not counting references.
- **Group members:** List the full name and JHU email of every member at the top.
- **Structure:** Include the **three sections** below.

Sections

1. **Motivation.** What problem are you tackling, and why does it matter for machine programming? Explain what your project adds beyond existing work, and briefly mention the closest related papers or tools.
2. **Expected Outcomes.** What do you expect to deliver, and how will you know it works? Describe the metrics, benchmarks, or case studies you will use, and what you will compare against.
3. **Division of Work.** Say briefly which part of the project each member is responsible for.

Template

```
# <Project Title>
**Members:** Alice Smith (asmith1@jh.edu), Bob Lee (blee2@jh.edu), ...

## 1. Motivation
## 2. Expected Outcomes
## 3. Division of Work
```

When submitting on Gradescope, one member uploads the `.md` file and adds all teammates to the submission.

News bit of the day...

October 26, 2026 Research Publication

Working with academic communities to build scientific understanding

OpenAI Research Products Business Developers Company Foundation

October 26, 2026

Working w
communi
scientific u

We are announcing three new discoveries in mathematics, computer science, and physics about harmonic analysis, complexity theory, and quantum field theory.

Our release process continues to evolve in response to feedback from the associated academic communities. In our view, these manuscripts are best understood as a form of raw research data. To convert this raw research data into scientific understanding, in this release we are experimenting with a new approach. We reached out to external experts and asked them to act as shepherds, organizing groups to understand the new results. They advised three different release strategies:

- For the math result, release the AI output, leaving researchers free to organize their own efforts to explain the result.
- For the computer science result, form an external group of collaborators to explain the result, while making the AI output publicly available for anyone to examine and build on.
- For the physics result, form an external group of collaborators and defer to them on the release of materials.

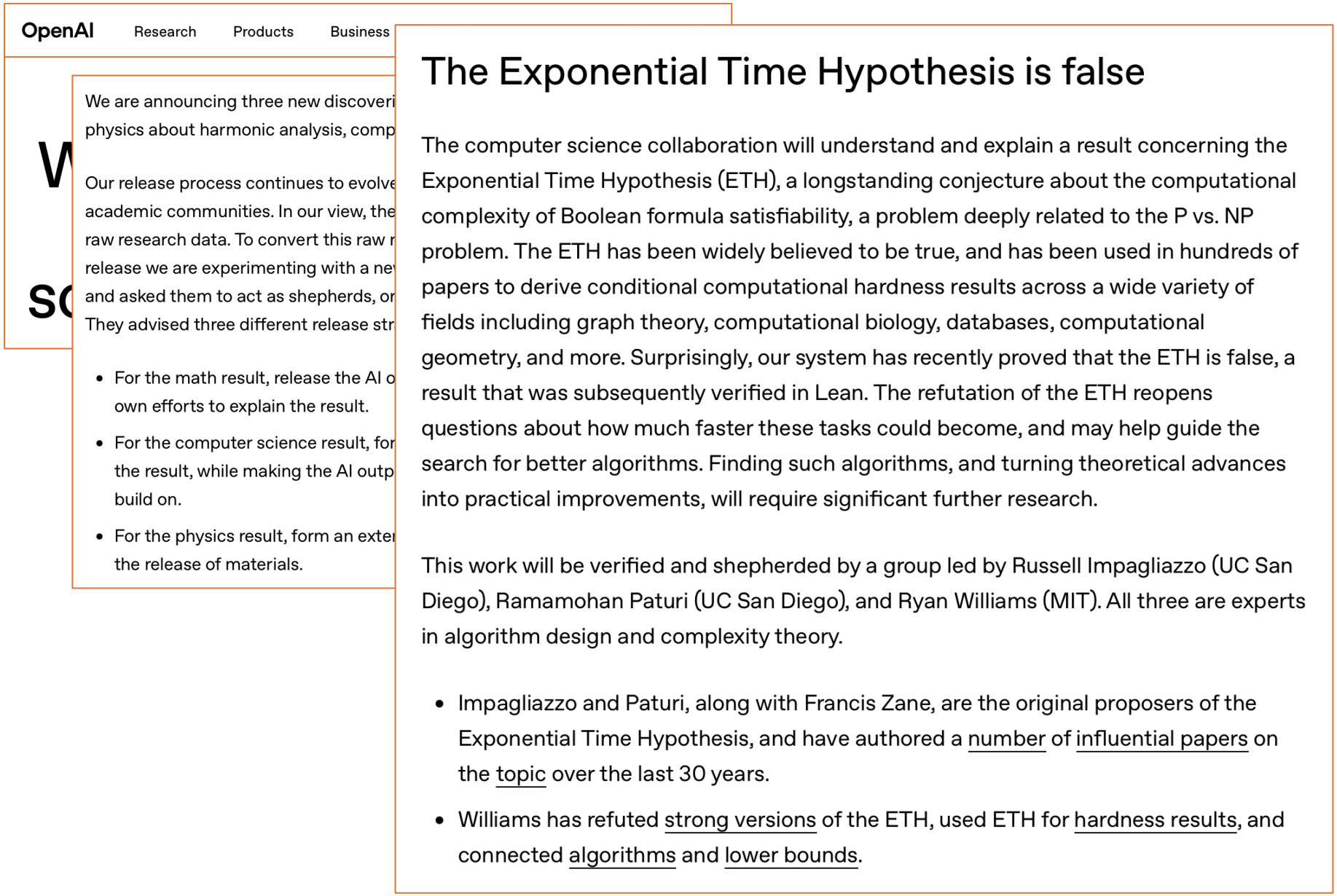
W

SC

We are announcing three new discoveries in mathematics, computer science, and physics about harmonic analysis, complexity theory, and quantum field theory.

Our release process continues to evolve in response to feedback from the associated academic communities. In our view, these manuscripts are best understood as a form of raw research data. To convert this raw research data into scientific understanding, in this release we are experimenting with a new approach. We reached out to external experts and asked them to act as shepherds, organizing groups to understand the new results. They advised three different release strategies:

- For the math result, release the AI output, leaving researchers free to organize their own efforts to explain the result.
- For the computer science result, form an external group of collaborators to explain the result, while making the AI output publicly available for anyone to examine and build on.
- For the physics result, form an external group of collaborators and defer to them on the release of materials.


OpenAI
Research
Products
Business
We are announcing three new discoveri
physics about harmonic analysis, comp
Our release process continues to evolve
academic communities. In our view, the
raw research data. To convert this raw r
release we are experimenting with a ne
and asked them to act as shepherds, or
They advised three different release str
For the math result, release the AI o
own efforts to explain the result.
For the computer science result, for
the result, while making the AI outp
build on.
For the physics result, form an exte
the release of materials.
The Exponential Time Hypothesis is false
The computer science collaboration will understand and explain a result concerning the Exponential Time Hypothesis (ETH), a longstanding conjecture about the computational complexity of Boolean formula satisfiability, a problem deeply related to the P vs. NP problem. The ETH has been widely believed to be true, and has been used in hundreds of papers to derive conditional computational hardness results across a wide variety of fields including graph theory, computational biology, databases, computational geometry, and more. Surprisingly, our system has recently proved that the ETH is false, a result that was subsequently verified in Lean. The refutation of the ETH reopens questions about how much faster these tasks could become, and may help guide the search for better algorithms. Finding such algorithms, and turning theoretical advances into practical improvements, will require significant further research.
This work will be verified and shepherded by a group led by Russell Impagliazzo (UC San Diego), Ramamohan Paturi (UC San Diego), and Ryan Williams (MIT). All three are experts in algorithm design and complexity theory.
Impagliazzo and Paturi, along with Francis Zane, are the original proposers of the Exponential Time Hypothesis, and have authored a number of influential papers on the topic over the last 30 years.
Williams has refuted strong versions of the ETH, used ETH for hardness results, and connected algorithms and lower bounds.

OpenAI Research Products Business Developers Company Foundation

We are announcing three new discoveries in mathematics, computer science, and physics about harmonic analysis, complexity theory, and quantum field theory.

Our rele
acaden
raw res
release
and ask
They ac

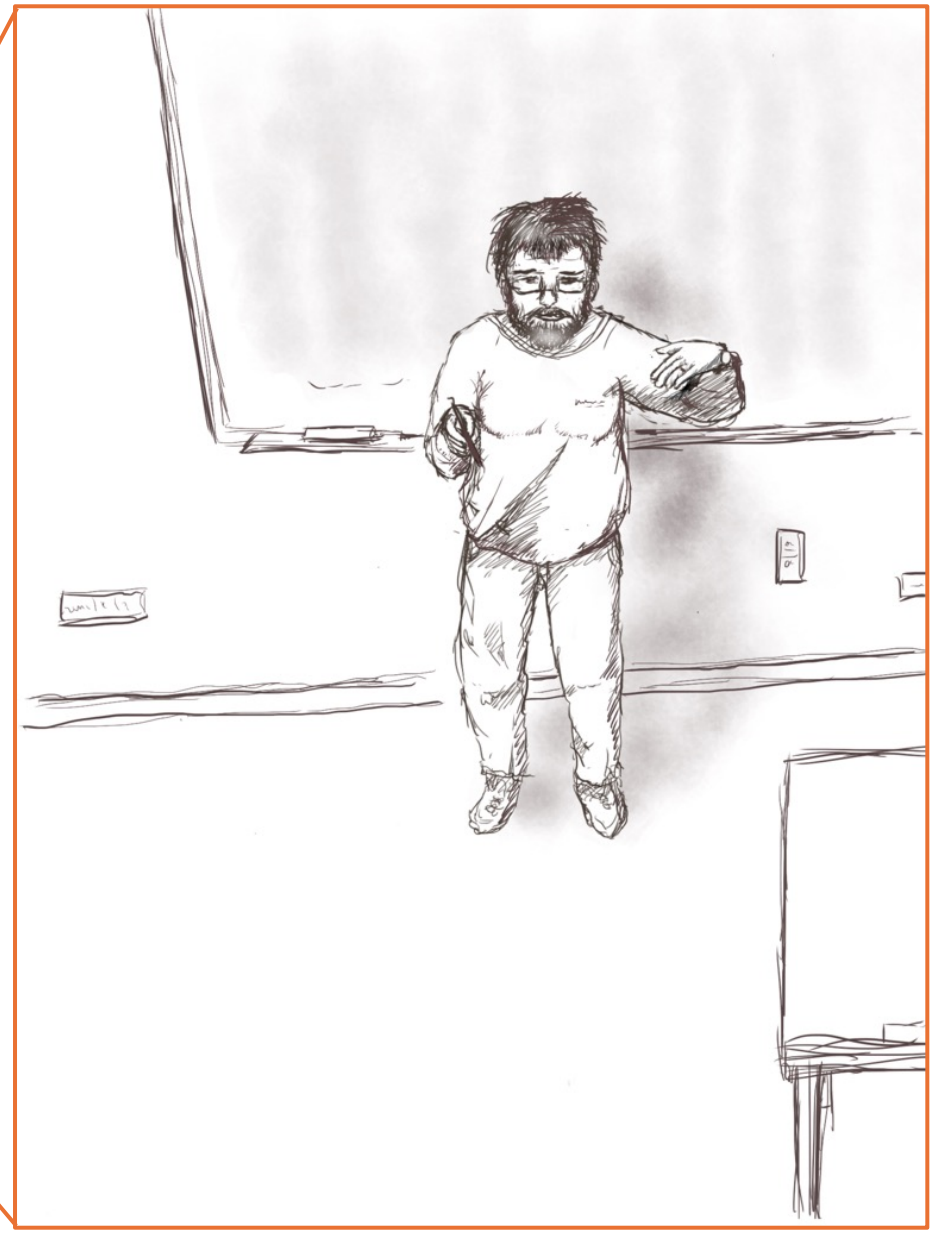
- For
own
- For
the
buil
- For
the

The Exponential Time Hypothesis is false

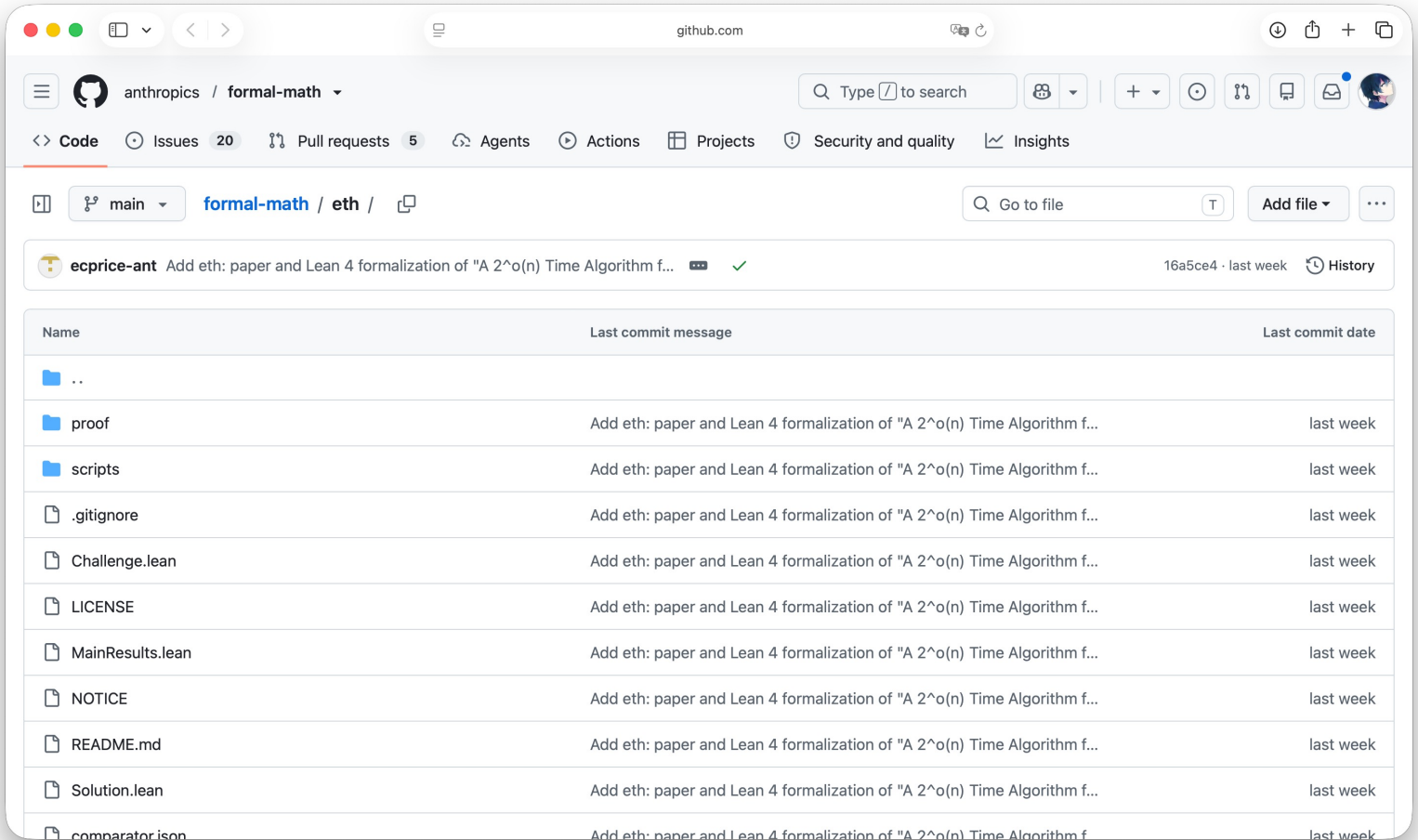
The computer science collaboration will understand and explain a result concerning the Exponential Time Hypothesis (ETH), a longstanding conjecture about the computational complexity of Boolean formula satisfiability, a problem deeply related to the P vs. NP problem. The ETH has been widely believed to be true, and has been used in hundreds of papers to derive conditional computational hardness results across a wide variety of fields including graph theory, computational biology, databases, computational geometry, and more. Surprisingly, our system has recently proved that the ETH is false, a result that was subsequently verified in Lean. The refutation of the ETH reopens questions about how much faster these tasks could become, and may help guide the search for better algorithms. Finding such algorithms, and turning theoretical advances into practical improvements, will require significant further research.

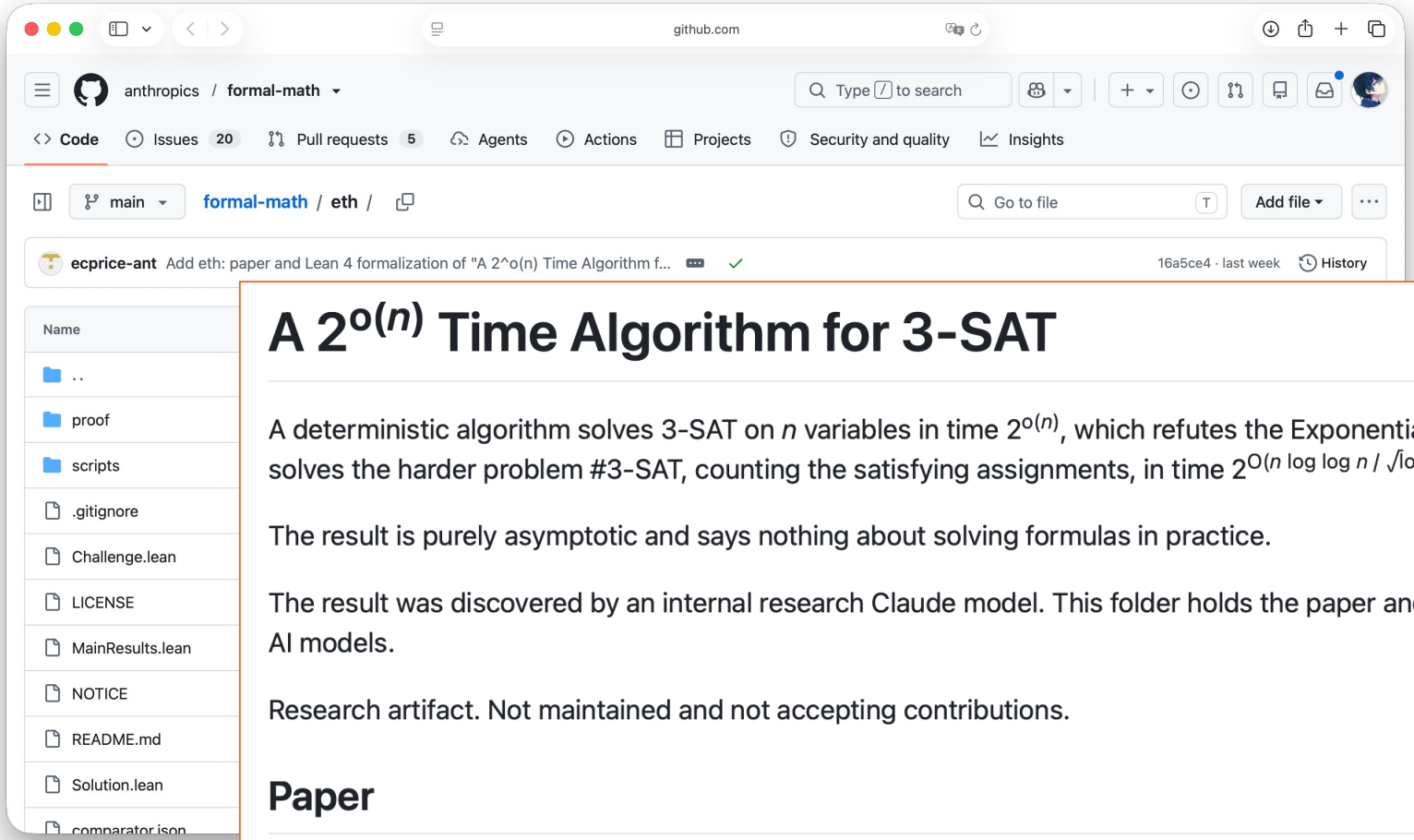
This work will be verified and shepherded by a group led by Russell Impagliazzo (UC San Diego), Ramamohan Paturi (UC San Diego), and Ryan Williams (MIT). All three are experts in algorithm design and complexity theory.

- Impagliazzo and Paturi, along with Francis Zane, are the original proposers of the Exponential Time Hypothesis, and have authored a number of influential papers on the topic over the last 30 years.
- Williams has refuted strong versions of the ETH, used ETH for hardness results, and connected algorithms and lower bounds.



Ziyang Li, 2016


github.com
anthropics / formal-math
Type / to search
Code
Issues 20
Pull requests 5
Agents
Actions
Projects
Security and quality
Insights
main
formal-math / eth /
Go to file T
Add file
ecprice-ant Add eth: paper and Lean 4 formalization of "A 2^o(n) Time Algorithm f...
16a5ce4 · last week
History
Name
Last commit message
Last commit date
..
proof
Add eth: paper and Lean 4 formalization of "A 2^o(n) Time Algorithm f...
last week
scripts
Add eth: paper and Lean 4 formalization of "A 2^o(n) Time Algorithm f...
last week
.gitignore
Add eth: paper and Lean 4 formalization of "A 2^o(n) Time Algorithm f...
last week
Challenge.lean
Add eth: paper and Lean 4 formalization of "A 2^o(n) Time Algorithm f...
last week
LICENSE
Add eth: paper and Lean 4 formalization of "A 2^o(n) Time Algorithm f...
last week
MainResults.lean
Add eth: paper and Lean 4 formalization of "A 2^o(n) Time Algorithm f...
last week
NOTICE
Add eth: paper and Lean 4 formalization of "A 2^o(n) Time Algorithm f...
last week
README.md
Add eth: paper and Lean 4 formalization of "A 2^o(n) Time Algorithm f...
last week
Solution.lean
Add eth: paper and Lean 4 formalization of "A 2^o(n) Time Algorithm f...
last week
comparator.json
Add eth: paper and Lean 4 formalization of "A 2^o(n) Time Algorithm f
last week



A $2^{o(n)}$ Time Algorithm for 3-SAT

A deterministic algorithm solves 3-SAT on n variables in time $2^{o(n)}$, which refutes the Exponential Time Hypothesis (ETH). The algorithm in fact solves the harder problem #3-SAT, counting the satisfying assignments, in time $2^{O(n \log \log n / \sqrt{\log n})}$.

The result is purely asymptotic and says nothing about solving formulas in practice.

The result was discovered by an internal research Claude model. This folder holds the paper and its Lean 4 formalization, which was written by AI models.

Research artifact. Not maintained and not accepting contributions.

Paper

paper.pdf: *A $2^{o(n)}$ Time Algorithm for 3-SAT*, by Claude (Anthropic). Hexagon link to appear.

Galactus the Devourer

On efficient algorithms, the Exponential Time Hypothesis, and losing the plot.

BEN RECHT

OCT 30, 2026

122 17 20 Share

Galactus

On efficient algor

BEN RECHT

OCT 30, 2026

122 17

algorithm that required a quartic number of operations. To later formalists like Richard Karp, it seemed like a “polynomial” number of operations was a reasonable definition for goodness. Polynomial time is good. Exponential time is bad. This created the field of complexity theory, with the goal of proving that all sorts of algorithms run in polynomial time and characterizing the problems for which no polynomial-time algorithm existed.

But polynomial time probably haunted Edmonds. It is not a “good” algorithm if your procedure scales like n^{1000}, even if this is technically a polynomial. More absurdly,

$$n^{10^{80}}$$

is a polynomial bigger than the exponential function 2^n for any n we could ever encounter. Polynomial was always an imperfect definition of good, but we ran with it. Thousands of papers later, we just have a zoo of confusing complexity results unmoored from the realities of computation.

algorithm that required a quartic number of operations. To later formalists like Richard Karp, it seemed like a “polynomial” number of operations was a reasonable definition for goodness. Po
of complexity the
polynomial time a
algorithm existed

But polynomial ti
procedure scales

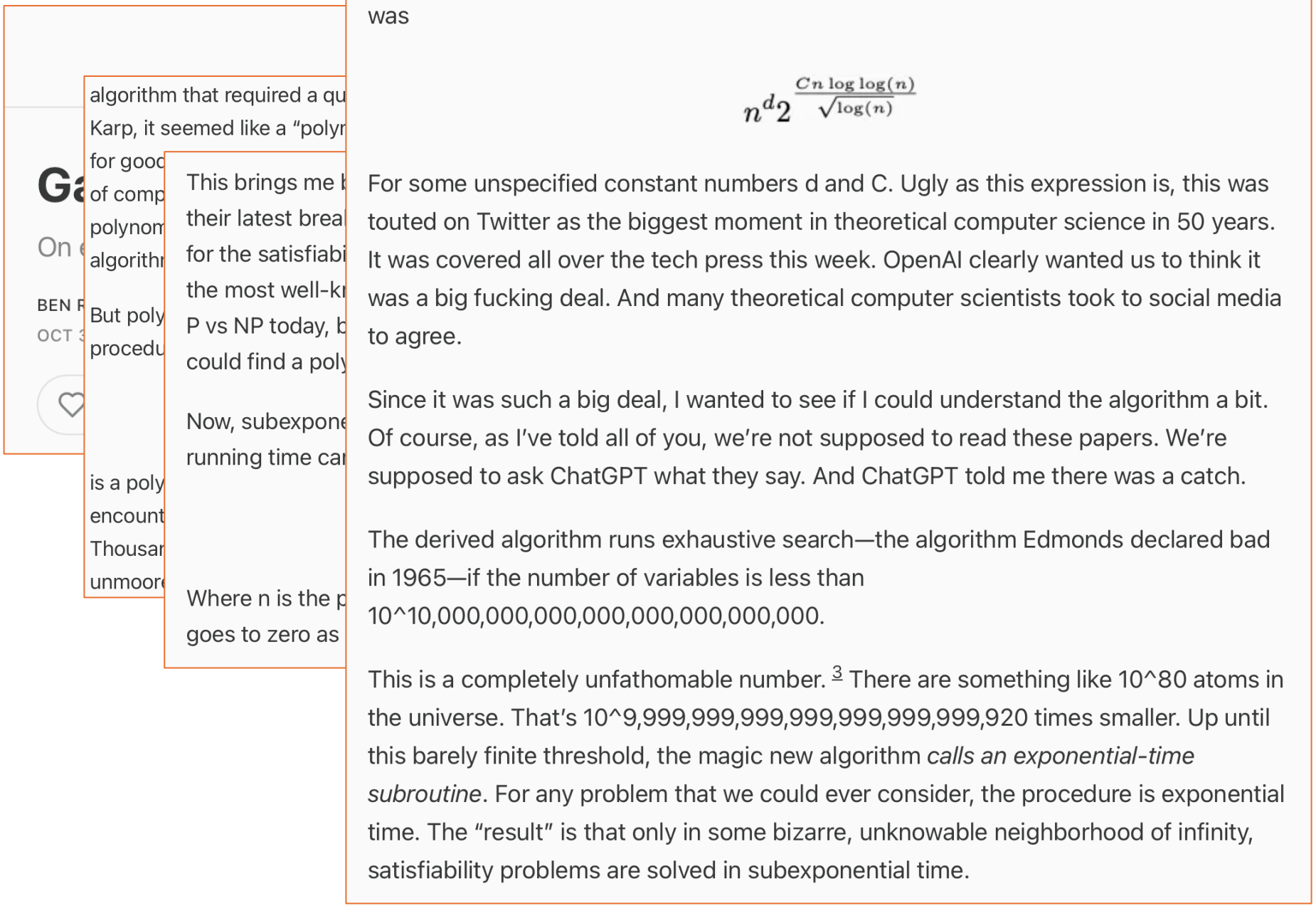
is a polynomial bi
encounter. Polyn
Thousands of pa
unmoored from t

This brings me back to our beneficent friends at OpenAI. This week they declared their latest breakthrough in theoretical computer science. They provided an algorithm for the satisfiability problem that ran in “subexponential time.” Satisfiability is one of the most well-known “NP complete problems, and I don’t want to go into the details of P vs NP today, but suffice it to say that you would win a Clay Millennium Prize if you could find a polynomial-time algorithm for satisfiability. [2]

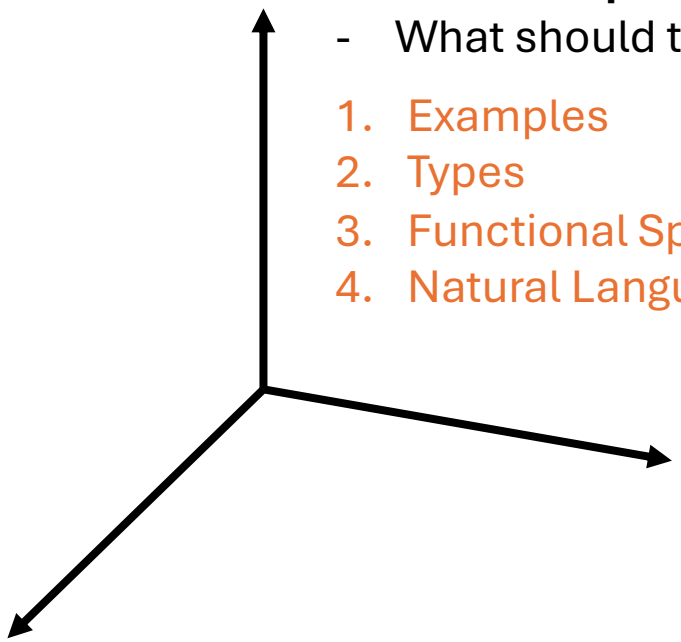
Now, subexponential time does not mean polynomial time. It merely means that the running time can be bounded as

$$2^{f(n)}$$

Where n is the problem size, and f is a “sublinear” function, meaning the limit of f(n)/n goes to zero as n goes to infinity.


algorithm that required a qu
Karp, it seemed like a “polyn
for good
of comp
polynom
algorithm
But poly
procedu
is a poly
encount
Thousan
unmoore
This brings me b
their latest breal
for the satisfiabi
the most well-kn
P vs NP today, b
could find a poly
Now, subexpone
running time can
Where n is the p
goes to zero as
Indeed, in the case of OpenAI’s new algorithm for the satisfiability problem, the bound was
$$n^d 2^{\frac{Cn \log\log(n)}{\sqrt{\log(n)}}}$$
For some unspecified constant numbers d and C. Ugly as this expression is, this was touted on Twitter as the biggest moment in theoretical computer science in 50 years. It was covered all over the tech press this week. OpenAI clearly wanted us to think it was a big fucking deal. And many theoretical computer scientists took to social media to agree.
Since it was such a big deal, I wanted to see if I could understand the algorithm a bit. Of course, as I’ve told all of you, we’re not supposed to read these papers. We’re supposed to ask ChatGPT what they say. And ChatGPT told me there was a catch.
The derived algorithm runs exhaustive search—the algorithm Edmonds declared bad in 1965—if the number of variables is less than 10^10,000,000,000,000,000,000,000,000.
This is a completely unfathomable number. [3] There are something like 10^80 atoms in the universe. That’s 10^9,999,999,999,999,999,999,999,920 times smaller. Up until this barely finite threshold, the magic new algorithm *calls an exponential-time subroutine*. For any problem that we could ever consider, the procedure is exponential time. The “result” is that only in some bizarre, unknowable neighborhood of infinity, satisfiability problems are solved in subexponential time.

The Course So Far



Behavioral Specification
- What should the program do?

1. Examples
2. Types
3. Functional Specifications
4. Natural Language

Structural Specification
- What is the space of the programs?

General Purpose Programming Language
Python / Java / C / Rust / …

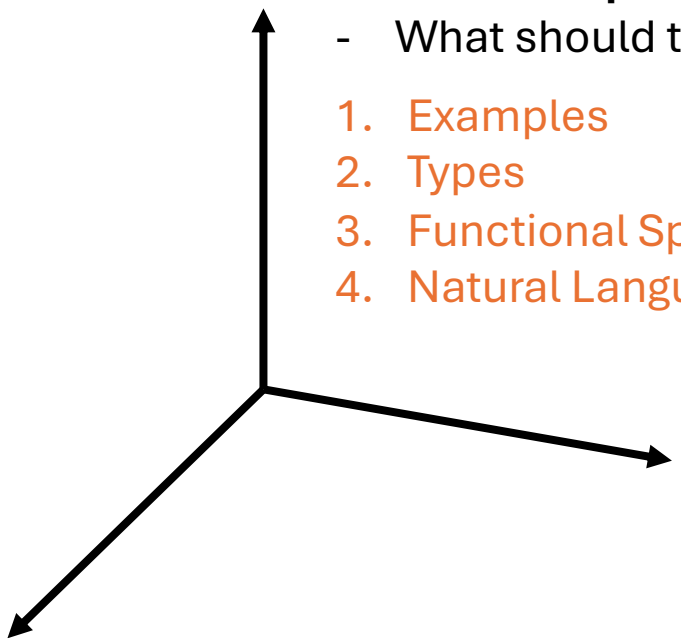
Custom Domain Specific Languages

Synthesis Strategy
- How do we find such a program?

Enumeration: Bottom-up and Top-Down

Programming with Large Language Models
- Next token prediction, prompting, controlled decoding
- Iterative refinement, agentic frameworks and tool use
- Pre-training, fine-tuning, reinforcement learning

Considering software quality...



Behavioral Specification

- What should the program do?

1. Examples
2. Types
3. Functional Specifications
4. Natural Language

Correctness Specifications

Structural Specification

- What is the space of the programs?

General Purpose Programming Language

Python / Java / C / Rust / ...

Fixed

Custom Domain Specific Languages

Synthesis Strategy

- How do we find such a program?

Enumeration: Bottom-up and Top-Down

Programming with Large Language Models

- Next token prediction, prompting, controlled decoding
- Iterative refinement, agentic frameworks and tool use
- Pre-training, fine-tuning, reinforcement learning

Fixed

Considering software quality...

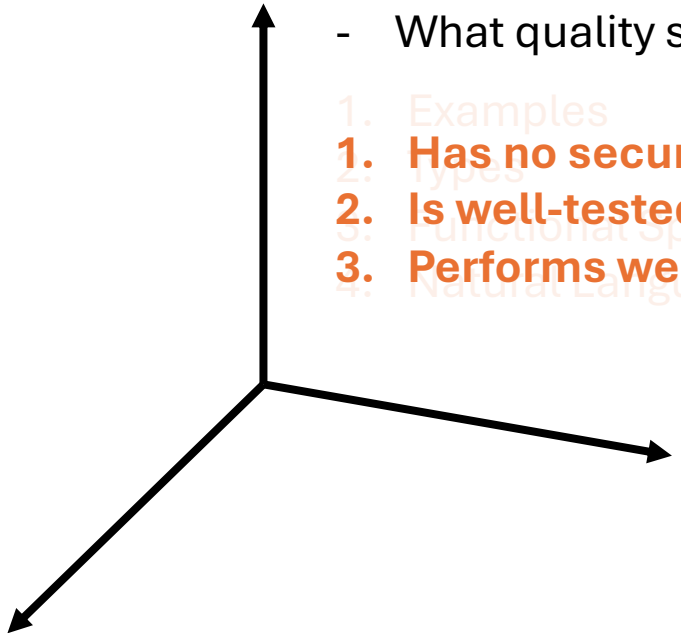
Quality Specifications

Behavioral Specification

- What quality should programs have?

1. Examples
2. Types
3. Functional Specifications
4. Natural Language

1. **Has no security flaw** (safety, privacy, etc.)
2. **Is well-tested** (no regression, good coverage, etc.)
3. **Performs well** (fast, use less resource, etc.)



Synthesis Strategy

- How do we find such a program?

Enumeration: Bottom-up and Top-Down

Programming with Large Language Models

- Next token prediction, prompting, controlled decoding
- Iterative refinement, agentic frameworks and tool use
- Pre-training, fine-tuning, reinforcement learning

Fixed

Structural Specification

- What is the space of the programs?

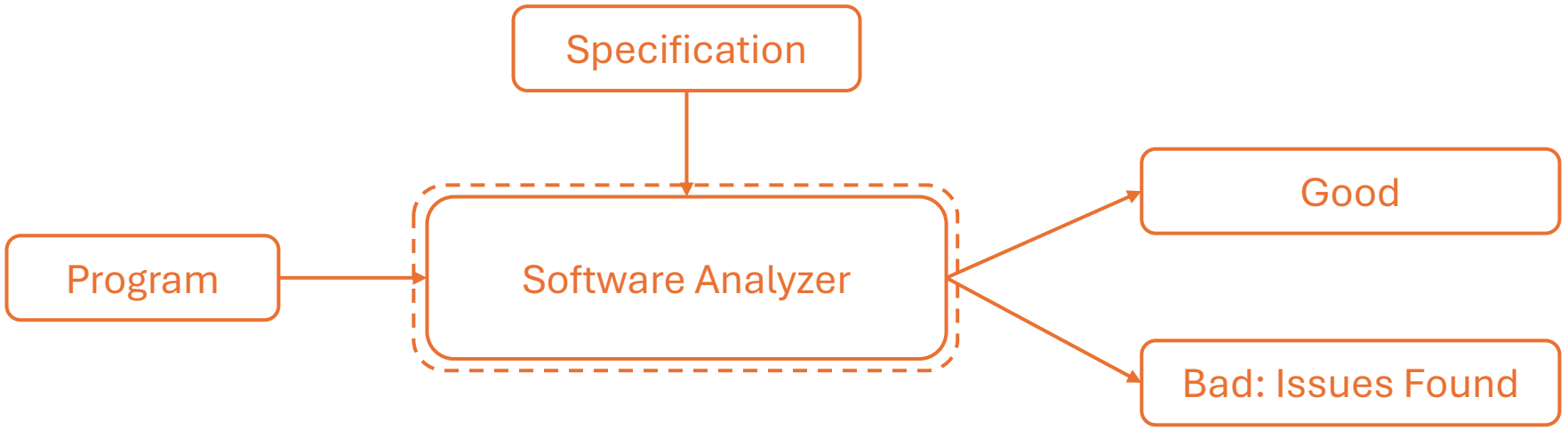
General Purpose Programming Language

Python / Java / C / Rust / ...

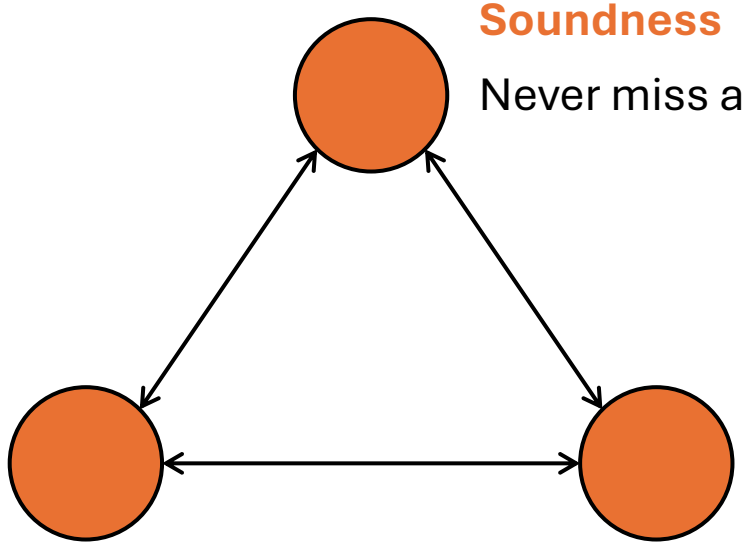
Custom Domain Specific Languages

Fixed

Software Analysis



Software Analysis: Impossible Triangle



Soundness

Never miss a real bug

Completeness

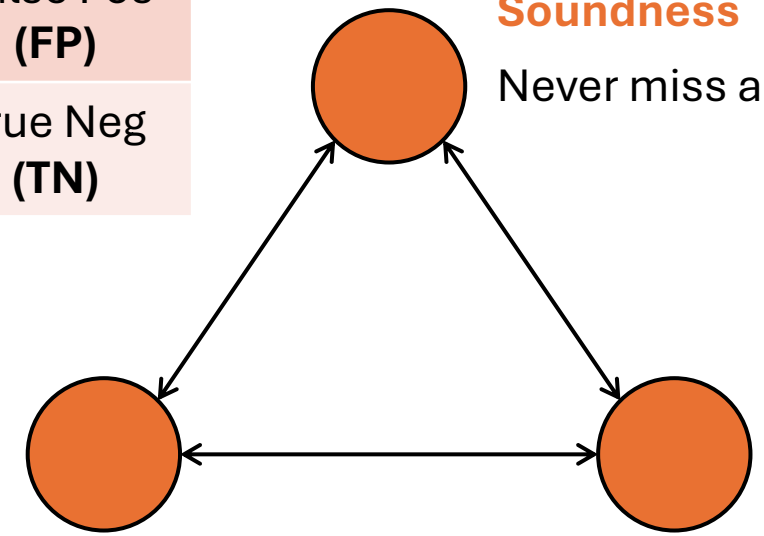
Never raise a false alarm

Termination

Always finish the analysis within finite time

Software Analysis: Impossible Triangle

	Real Bug	Not a Bug
Reported (positive)	True Pos **(TP)**	False Pos **(FP)**
Not Reported (negative)	False Neg **(FN)**	True Neg **(TN)**



Soundness

Never miss a real bug

Completeness

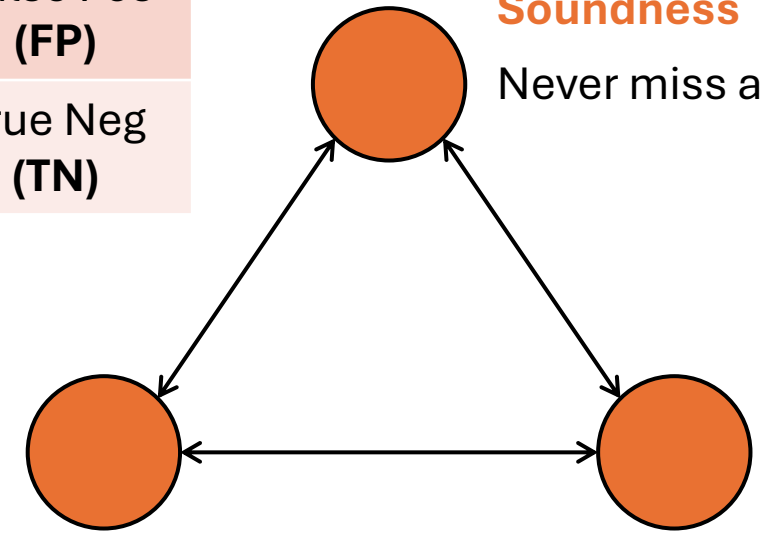
Never raise a false alarm

Termination

Always finish the analysis within finite time

Software Analysis: Impossible Triangle

	Real Bug	Not a Bug
Reported (positive)	True Pos **(TP)**	False Pos **(FP)**
Not Reported (negative)	False Neg **(FN)**	True Neg **(TN)**



Soundness #FN = 0

Never miss a real bug

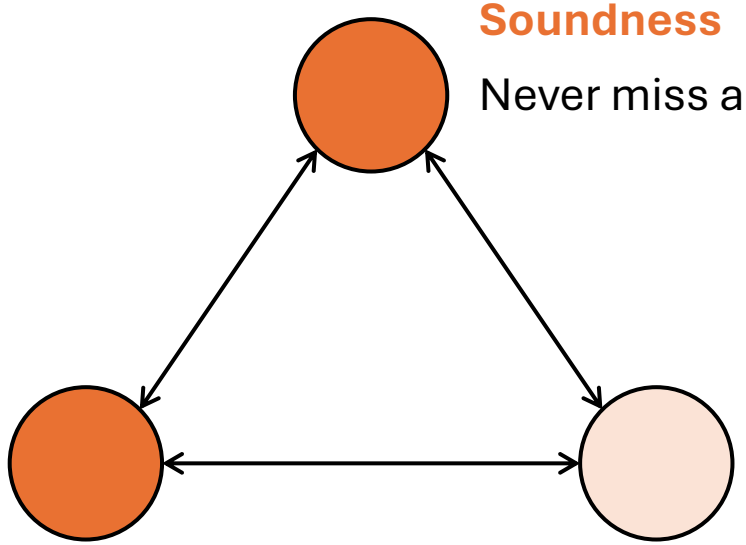
#FP = 0 **Completeness**

Never raise a false alarm

Termination

Always finish the analysis within finite time

Software Analysis: Impossible Triangle



Soundness

Never miss a real bug

Completeness

Never raise a false alarm

~~**Termination**~~

~~Always finish the analysis within finite time~~

Any judgement is precise and accurate, but may not terminate

Formal Verification:

Automated Theorem Proving

Interactive Theorem Proving

Software Analysis: Impossible Triangle

Dynamic Analysis:

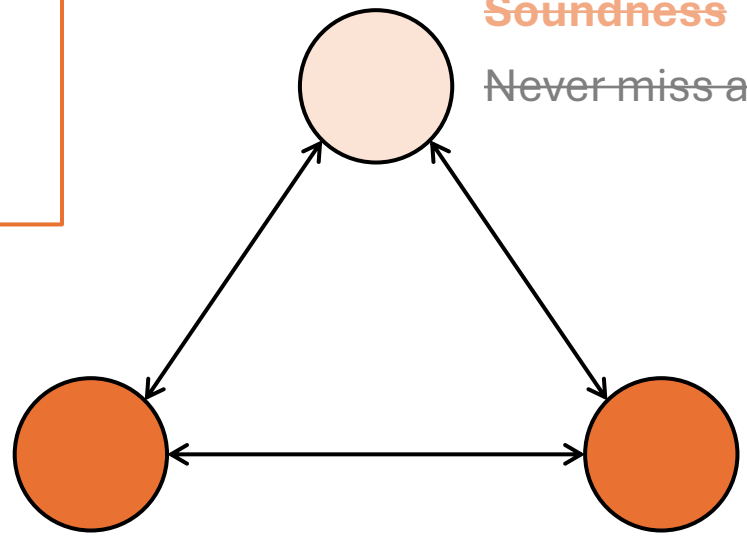
Unit Testing

- (assuming tests are correct)

Fuzzing

- (assuming harness is real)

The unit test cases might not be comprehensive enough.



~~**Soundness**~~

~~Never miss a real bug~~

Completeness

Never raise a false alarm

Termination

Always finish the analysis within finite time

Software Analysis: Impossible Triangle

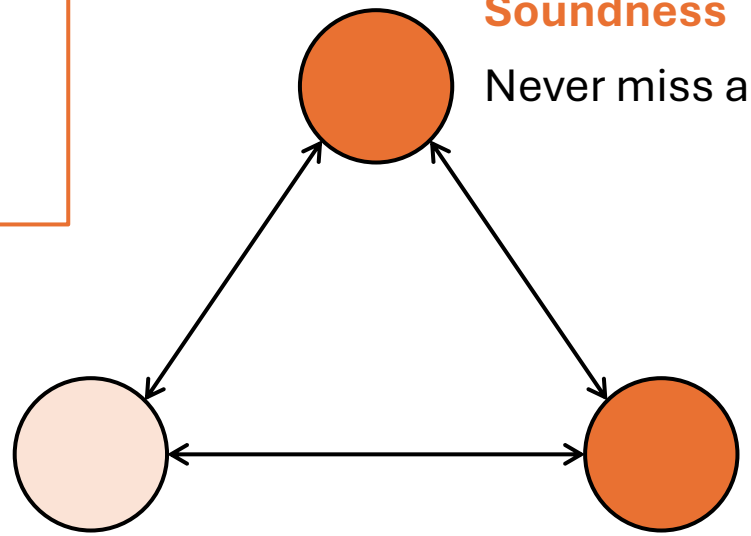
Static Analysis:

Taint Analysis

- (assuming correct specs)

Abstract Interpretation

- (assuming correct domains)



Soundness

Never miss a real bug

~~**Completeness**~~

~~Never raise a false alarm~~

Termination

Always finish the analysis within finite time

May report an alarm for developer confirmation, and that is rejected

Software Analysis: Impossible Triangle

	Real Bug	Not a Bug
Reported (positive)	True Pos **(TP)**	False Pos **(FP)**
Not Reported (negative)	False Neg **(FN)**	True Neg **(TN)**

Soundness #FN = 0

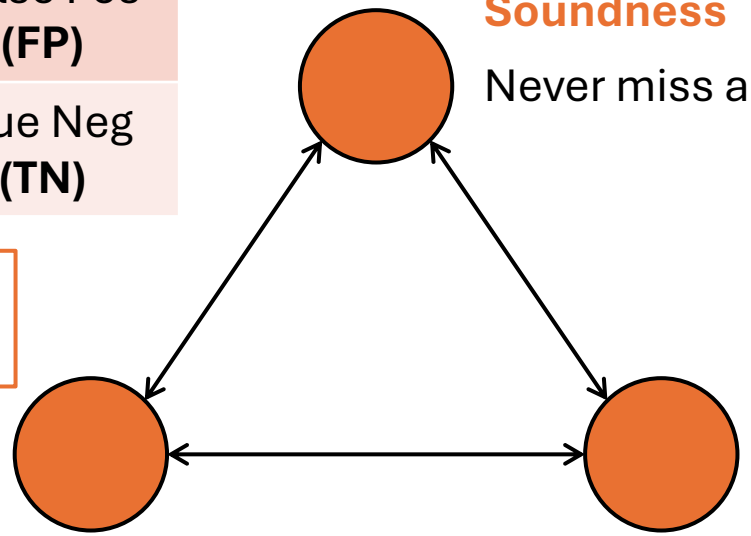
Never miss a real bug

#FN is small: Analysis achieves high **"Recall"**

#FP is small: Analysis achieves high **"Precision"**

#FP = 0 **Completeness**

Never raise a false alarm



Termination

Always finish the analysis within finite time

Forms of Software Analysis

Static Analysis

Dynamic Analysis

Forms of Software Analysis

Static Analysis

Examine code without concrete execution

Dynamic Analysis

Observes program behavior while executing

Forms of Software Analysis

Static Analysis

Examine code without concrete execution

- Taint analysis
- Reachability analysis
- Abstract interpretation
- Symbolic execution
- ...

Dynamic Analysis

Observes program behavior while executing

- Unit testing
- Fuzzing
- Property-based testing
- Penetration testing
- ...

Dynamic Analysis

- Some bugs only manifest when code runs
  - Runtime errors
  - Unhandled exceptions
  - Errors dependent on external environments
- Want to concretely execute code
  - Observe program behavior
  - Analyze potential mistakes

Dynamic Testing

Unit Testing

Human-written tests in the form of concrete input-output

```
assert f(2) == 4
```

Dynamic Testing

Unit Testing

Human-written tests in the form of concrete input-output

```
assert f(2) == 4
```

Fuzzing

Randomly generating test inputs

```
for i in 0..10^4:
  x = gen_input()
  f(x)
```

Dynamic Testing

Unit Testing

Human-written tests in the form of concrete input-output

```
assert f(2) == 4
```

Fuzzing

Randomly generating test inputs

```
for i in 0..10^4:
  x = gen_input()
  f(x)
```

Property-based testing

Randomly generating test inputs, but with properties checking outputs

```
for i in 0..10^4:
  x = gen_input()
  y = f(x)
  assert property(x,y)
```

Dynamic Testing

Unit Testing

Human-written tests in the form of concrete input-output

```
assert f(2) == 4
```

Fuzzing

Randomly generating test inputs

```
for i in 0..10^4:
  x = gen_input()
  f(x)
```

Property-based testing

Randomly generating test inputs, but with properties checking outputs

```
for i in 0..10^4:
  x = gen_input()
  y = f(x)
  assert property(x,y)
```

Penetration testing

Crafting adversarial inputs to trigger vulnerabilities rather than getting outputs

```
try:
  f(very_bad_input)
  assert false
except:
  assert no_bad_behavior
```

Unit Testing

Human-written tests in the form of concrete input-output

```
assert f(2) == 4
```

Fuzzing

Randomly generating test inputs

```
for i in 0..10^4:
  x = gen_input()
  f(x)
```

Human Effort: figure out input-outputs, write the test cases

Property-based testing

Randomly generating test inputs, but with properties checking outputs

```
for i in 0..10^4:
  x = gen_input()
  y = f(x)
  assert property(x,y)
```

Penetration testing

Crafting adversarial inputs to trigger vulnerabilities rather than getting outputs

```
try:
  f(very_bad_input)
  assert false
except:
  assert no_bad_behavior
```

Human Effort: write the input generator and fuzzing harness

Unit Testing

Human-written tests in the form of concrete input-output

```
assert f(2) == 4
```

Fuzzing

Randomly generating test inputs

```
for i in 0..10^4:
  x = gen_input()
  f(x)
```

Human Effort: figure out input-outputs, write the test cases

Property-based testing

Randomly generating test inputs, but with properties checking outputs

```
for i in 0..10^4:
  x = gen_input()
  y = f(x)
  assert property(x,y)
```

Penetration testing

Crafting adversarial inputs to trigger vulnerabilities rather than getting outputs

```
try:
  f(very_bad_input)
  assert false
except:
  assert no_bad_behavior
```

Human Effort: write the input generator and fuzzing harness

Unit Testing

Human-written tests in the form of concrete input-output

```
assert f(2) == 4
```

Fuzzing

Randomly generating test inputs

```
for i in 0..10^4:
  x = gen_input()
  f(x)
```

Human Effort: figure out input-outputs, write the test cases

Property-based testing

Randomly generating test inputs, but with properties checking outputs

```
for i in 0..10^4:
  x = gen_input()
  y = f(x)
  assert property(x,y)
```

Human Effort: write the input generator, harness and the property

Penetration testing

Crafting adversarial inputs to trigger vulnerabilities rather than getting outputs

```
try:
  f(very_bad_input)
  assert false
except:
  assert no_bad_behavior
```

Human Effort: write the input generator and fuzzing harness

Human Effort: figure out the very bad input, describe the bad behavior

Unit Testing

Human-written tests in the form of concrete input-output

```
assert f(2) == 4
```

Fuzzing

Randomly generating test inputs

```
for i in 0..10^4:
  x = gen_input()
  f(x)
```

Human Effort: figure out input-outputs, write the test cases

Property-based testing

Randomly generating test inputs, but with properties checking outputs

```
for i in 0..10^4:
  x = gen_input()
  y = f(x)
  assert property(x,y)
```

Penetration testing

Crafting adversarial inputs to trigger vulnerabilities rather than getting outputs

```
try:
  f(very_bad_input)
  assert false
except:
  assert no_bad_behavior
```

Human Effort: write the input generator, harness and the property

Unit Testing: Precision Shots

- Human effort
  - Figure out expected input/output pairs (multiple)
  - Write a test case for each input/output pairs (multiple)
- Goal
  - Maximize confidence via coverage (approx.: line coverage)
  - Execute precise, human-chosen inputs that exercise specific code paths
  - Error handling is a path too!

Program – Line coverage: 0/5 (0%)

```
def calculate_interest(balance: float, months: int) -> float:
  rate = 0.02 # 2% per month
  if months > 12:
    rate = 0.03 # higher rate for long-term deposits
  return balance * rate / months
```

Program – Line coverage: 0/5 (0%)

```python
def calculate_interest(balance: float, months: int) -> float:
  rate = 0.02 # 2% per month
  if months > 12:
    rate = 0.03 # higher rate for long-term deposits
  return balance * rate / months
```

Test 1

```python
def test_short_term_interest():
  assert calculate_interest(1000, 6) == pytest.approx(3.33)
```

Program – Line coverage: 3/5 (60%)

```python
def calculate_interest(balance: float, months: int) -> float:
  rate = 0.02 # 2% per month
  if months > 12:
    rate = 0.03 # higher rate for long-term deposits
  return balance * rate / months
```

Test 1

```python
def test_short_term_interest():
  assert calculate_interest(1000, 6) == pytest.approx(3.33)
```

Program – Line coverage: 3/5 (60%)

```python
def calculate_interest(balance: float, months: int) -> float:
  rate = 0.02 # 2% per month
  if months > 12:
    rate = 0.03 # higher rate for long-term deposits
  return balance * rate / months
```

Test 1

```python
def test_short_term_interest():
  assert calculate_interest(1000, 6) == pytest.approx(3.33)
```

Test 2

```python
def test_long_term_interest():
  assert calculate_interest(1000, 24) == pytest.approx(1000 * 0.03 / 24)
```

Program – Line coverage: 5/5 (100%)

```python
def calculate_interest(balance: float, months: int) -> float:
  rate = 0.02 # 2% per month
  if months > 12:
    rate = 0.03 # higher rate for long-term deposits
  return balance * rate / months
```

Test 1

```python
def test_short_term_interest():
  assert calculate_interest(1000, 6) == pytest.approx(3.33)
```

Test 2

```python
def test_long_term_interest():
  assert calculate_interest(1000, 24) == pytest.approx(1000 * 0.03 / 24)
```

Are we done covering everything?

Program – Line coverage: 5/5 (100%)

```python
def calculate_interest(balance: float, months: int) -> float:
  rate = 0.02 # 2% per month
  if months > 12:
    rate = 0.03 # higher rate for long-term deposits
  return balance * rate / months
```

Test 1

```python
def test_short_term_interest():
  assert calculate_interest(1000, 6) == pytest.approx(3.33)
```

Test 2

```python
def test_long_term_interest():
  assert calculate_interest(1000, 24) == pytest.approx(1000 * 0.03 / 24)
```

Are we done covering everything?

Program – Line coverage: 5/5 (100%)

```
def calculate_interest(balance: float, months: int) -> float:
  rate = 0.02 # 2% per month
  if months > 12:
    rate = 0.03 # higher rate for long-term deposits
  return balance * rate / months
```

No!

Test 1

```
def test_short_term_interest():
  assert calculate_interest(1000, 6) == pytest.approx(3.33)
```

Test 2

```
def test_long_term_interest():
  assert calculate_interest(1000, 24) == pytest.approx(1000 * 0.03 / 24)
```

Are we done covering everything?

Program – Line coverage: 5/5 (100%)

```
def calculate_interest(balance: float, months: int) -> float:
  rate = 0.02 # 2% per month
  if months > 12:
    rate = 0.03 # higher rate for long-term deposits
  return balance * rate / months
```

No!

Test 1

```
def test_short_term_interest():
  assert calculate_interest(100
```

Test 2

```
def test_long_term_interest():
  assert calculate_interest(1000, 24) == pytest.approx(1000 * 0.03 / 24)
```

CWE-369: Divide By Zero

Weakness ID: 369
Vulnerability Mapping: ALLOWED
Abstraction: Base

Are we done covering everything?

Program – Line coverage: 5/5 (100%)

```
def calculate_interest(balance: float, months: int) -> float:
  rate = 0.02 # 2% per month
  if months > 12:
    rate = 0.03 # higher rate for long-term deposits
  return balance * rate / months
```

No!

Test 1

```
def test_short_term_interest():
  assert calculate_interest(1000, 6) == pytest.approx(3.33)
```

Test 2

```
def test_long_term_interest():
  assert calculate_interest(1000, 24) == pytest.approx(1000 * 0.03 / 24)
```

Test 3

```
def test_zero_months_divide_by_zero(): # covers implicit error path
  with pytest.raises(ZeroDivisionError):
    calculate_interest(1000, 0)
```

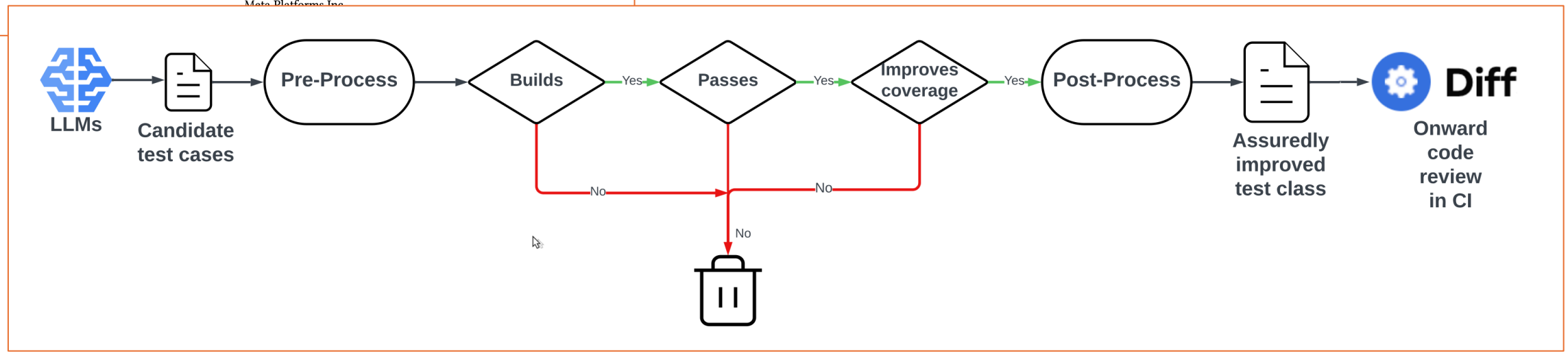
Automated Unit Test Improvement using Large Language Models at Meta

Nadia Alshahwan*
Jubin Chheda
Anastasia Finegenova
Beliz Gokkaya
Mark Harman
Inna Harper
Alexandru Marginean
Shubho Sengupta
Eddy Wang
Meta Platforms Inc.,
Menlo Park, California, USA

Automated Unit Test Improvement using Large Language Models at Meta, Alshahwan et. al., FSE 2024

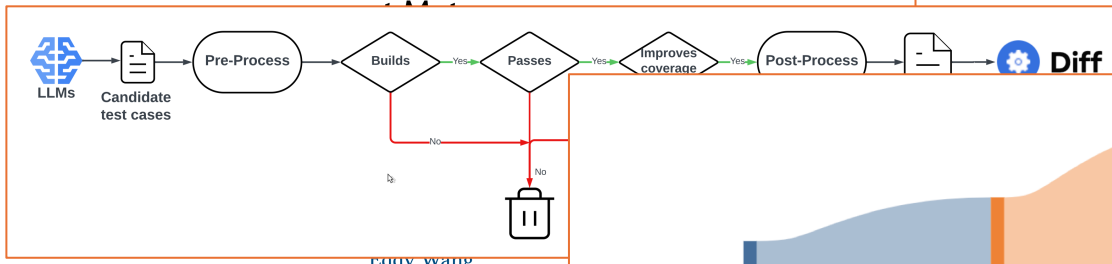
Automated Unit Test Improvement using Large Language Models at Meta

Nadia Alshahwan*
Jubin Chheda
Anastasia Finegenova
Beliz Gokkaya
Mark Harman
Inna Harper
Alexandru Marginean
Shubho Sengupta
Eddy Wang
Meta Platforms Inc



Automated Unit Test Improvement using Large Language Models at Meta, Alshahwan et. al., FSE 2024

Automated Unit Test Improvement using Large Language Models



Eddy Wang
Meta Platforms Inc.,
Menlo Park, California, USA

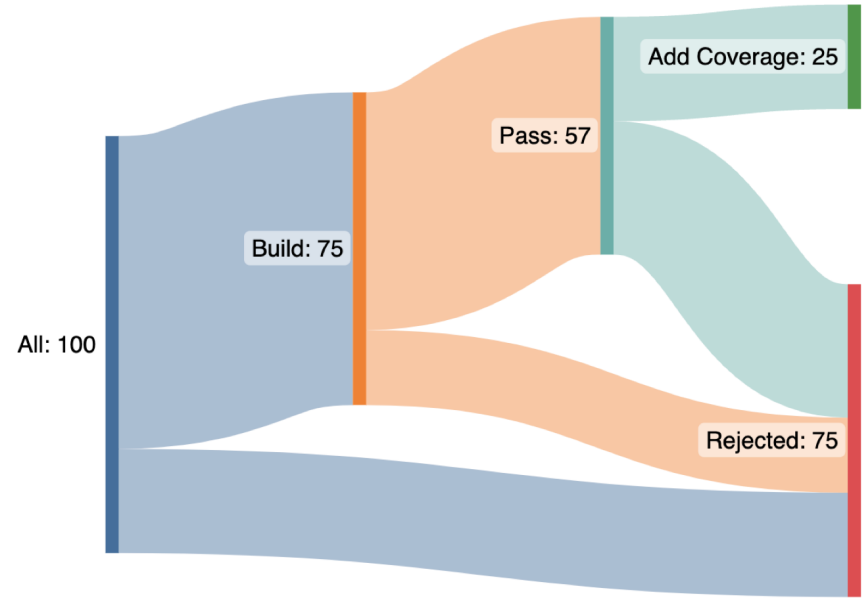


Figure 2: Sankey diagram showing the filtration process outcomes (as percentages of all test cases) from the Experimental Study on Instagram components for Reels and Stories products, using the four prompt strategies from Table 2 and the two language models, LLM1 and LLM2.

Automated Unit Test Improvement using Large Language Models at Meta, Alshahwan et. al., FSE 2024

Automated Unit Test



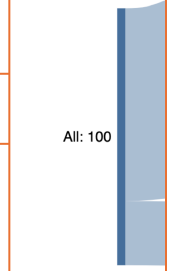


Figure 2: Sank
comes (as perc
Study on Insta
ucts, using the
two language

3.4.1 *Deployment at the Facebook App Test-a-thon December 2023.*

In this deployment, we had sufficient confidence to automatically submit recommendations from TestGen-LLM to engineers. There was no engineer pre-training process, no specific test-a-thon expectations, and no additional context provided to the engineers. This gave us a realistic assessment of the engineers' response to LLM-generated test recommendations provided 'out of the box'. Overall, over 50% of the diffs submitted were accepted by developers, a figure which rises to almost 70% of those which received a review by developers. Specifically, of the 280 diffs generated by TestGen-LLM:

- 144 were accepted by the engineer reviewing them.
- 64 were rejected and/or abandoned.
- 61 did not receive a review.
- 11 were withdrawn.

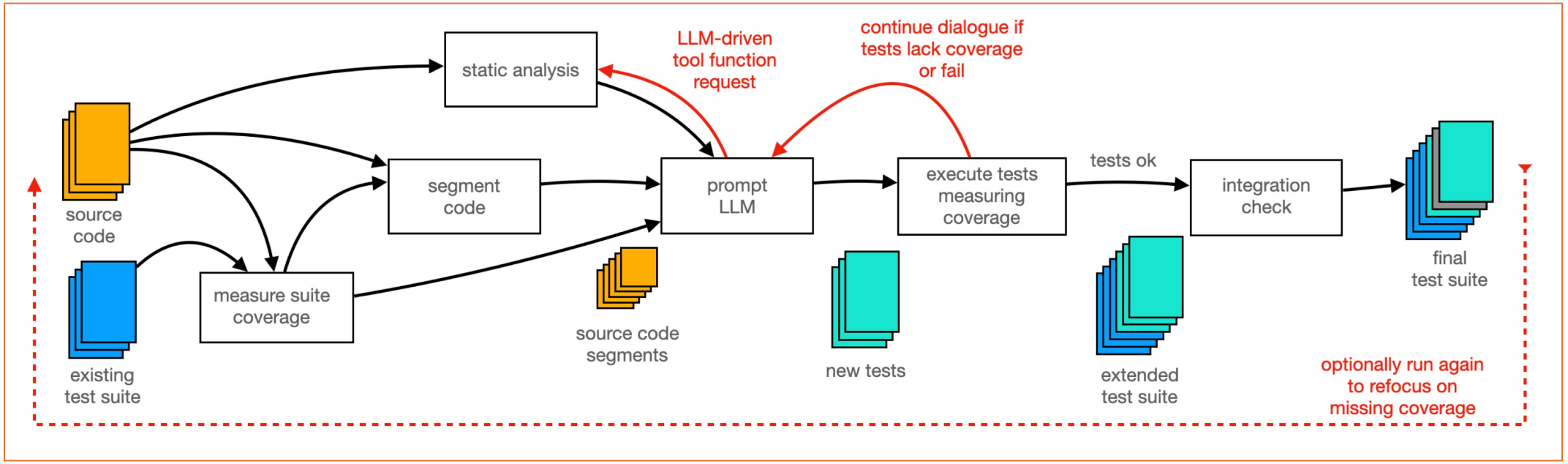
Automated Unit Test Improvement using Large Language Models at Meta, Alshahwan et. al., FSE 2024

CoverUp: Coverage-Guided LLM-Based Test Generation

JUAN ALTMAYER PIZZORNO, University of Massachusetts Amherst, United States
EMERY D. BERGER*, University of Massachusetts Amherst / Amazon, United States

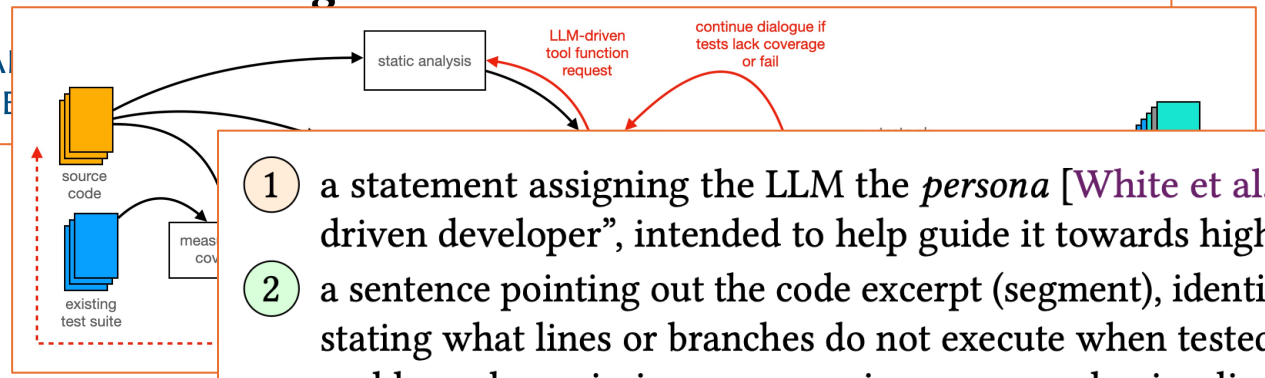
CoverUp: Coverage-Guided LLM-Based Test Generation

JUAN ALTMAYER PIZZORNO, University of Massachusetts Amherst, United States
EMERY D. BERGER*, University of Massachusetts Amherst / Amazon, United States



CoverUp: Coverage-Guided LLM-Based Test Generation

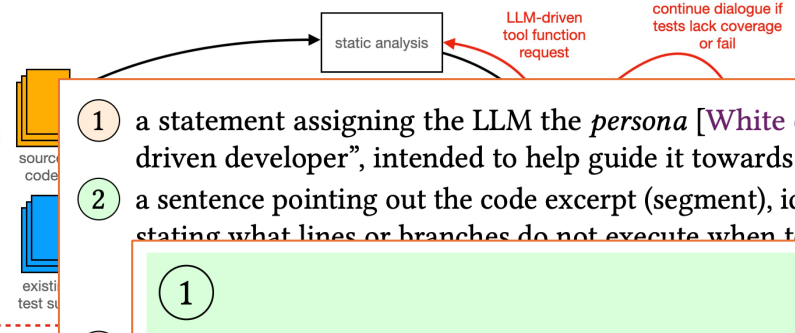
JUA
EME



1. a statement assigning the LLM the *persona* [White et al. 2023] of an "expert Python test-driven developer", intended to help guide it towards high quality tests;
2. a sentence pointing out the code excerpt (segment), identifying what file it comes from, and stating what lines or branches do not execute when tested. The portion specifying the lines and branches missing coverage is compressed using line ranges, simplifying the prompt and reducing token usage.
3. a request for `pytest` test functions and an encouragement for the LLM to use the provided tool function;
4. a series of other requests, such as "include assertions" and "avoid state pollution", to steer the result towards usable tests;
5. a request that the response only include the new Python tests to facilitate its extraction from the response and to reduce token usage; and
6. the code segment, prefixed by generated `import` statements and tagging the lines lacking (line or branch) coverage with their numbers.

CoverUp: Coverage-Guided LLM-Based Test Generation

JUA
EME



1 a statement assigning the LLM the *persona* [White et al. 2023] of an "expert Python test-driven developer", intended to help guide it towards high quality tests;

2 a sentence pointing out the code excerpt (segment), identifying what file it comes from, and stating what lines or branches do not execute when tested. The portion specifying the lines

3 ①

```
This test still lacks coverage: line 615 and branches 603->exit, 610->608, 618->exit
do not execute.
```

4 ②

5 6

```
Modify it to correct that; respond only with the complete Python code in backticks.
Use the get_info function as necessary.
```

Fig. 6. **Example of a coverage follow-up prompt:** CoverUp indicates to the LLM that a line and some branches still weren't covered (1), asking that it correct the test (2).

CoverUp: Coverage-Guided LLM-Based Test Generation

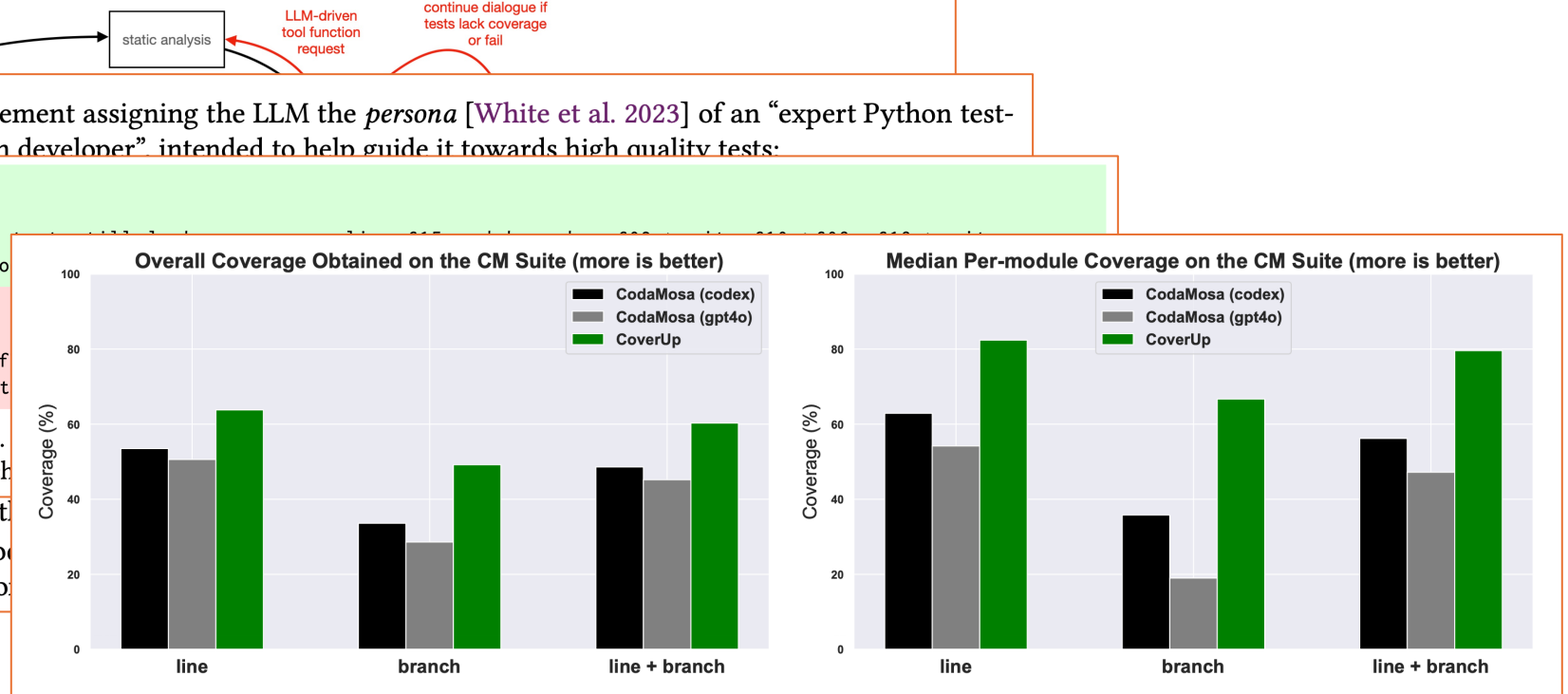


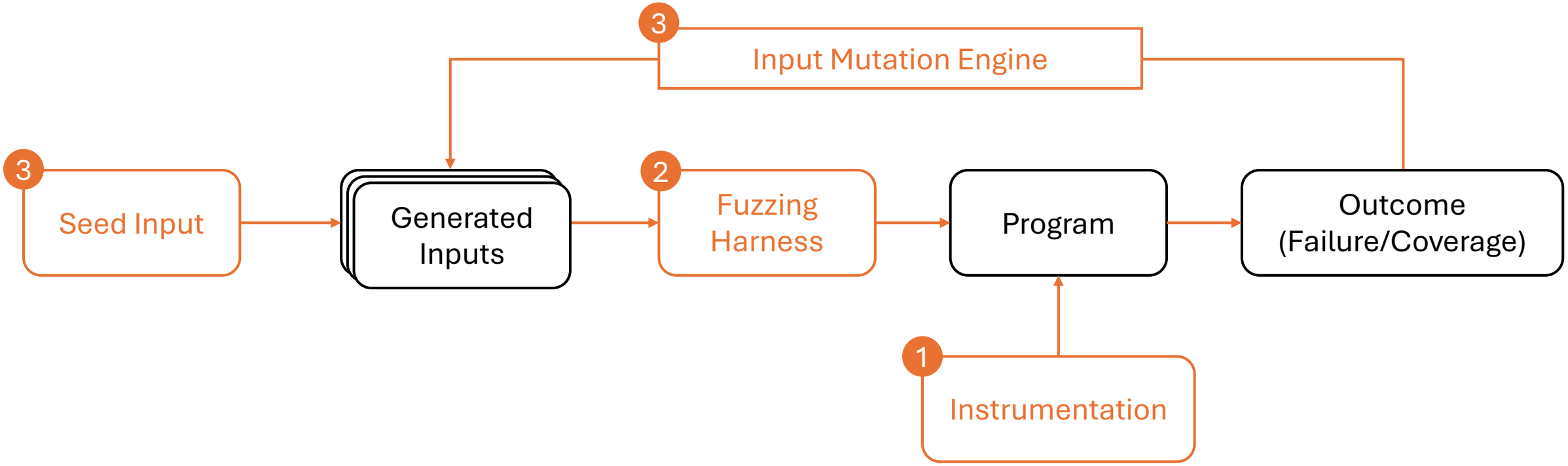
Fig. 12. **[RQ1] CoverUp yields higher overall and median per-module coverage:** Across the board, CoverUp yields higher coverage than CodaMosa, whether measured over the entire suite or on a module-by-module basis.

Fuzzing: Chaos with Purpose

```
for i in 0..10^4:
  x = gen_input()
  f(x)
```

- Goal
  - Explore huge input spaces automatically to trigger failures (crashes, exceptions, timeouts, sanitizer hits)
  - Unlike unit tests, fuzzing does not know the correct output. It detects incorrect behavior by symptoms
- Method
  - Massively and randomly generate inputs to programs

Fuzzing: The Loop



```c
int process_input(const uint8_t *data, size_t size) {
  if (size < 2) return 0; // need at least 2 bytes: cmd + length
  uint8_t cmd = data[0];
  uint8_t n = data[1]; // number of bytes to copy

  char buf[16]; // fixed-size buffer
  int secret = 0x41414141; // adjacent variable

  // If cmd == 0x42 we write to buffer
  if (cmd == 0x42) {
    // Write to buffer
    buf[n] = 0x00;
  }

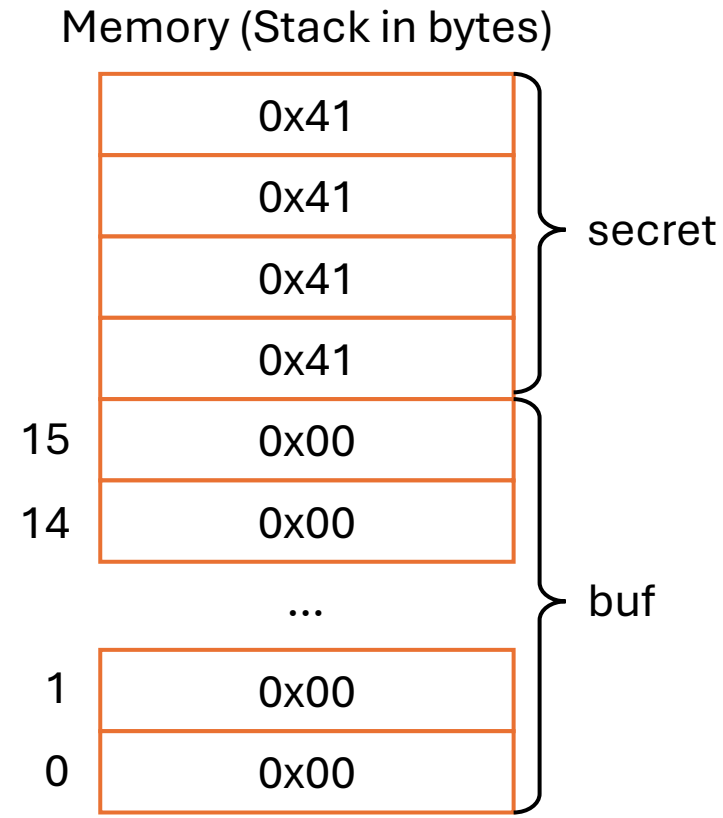
  // Return secret so program behavior may subtly change if corrupted
  return secret;
}
```

```c
int process_input(const uint8_t *data, size_t size) {
  if (size < 2) return 0; // need at least 2 bytes: cmd + length
  uint8_t cmd = data[0];
  uint8_t n = data[1]; // number of bytes to copy

  char buf[16]; // fixed-size buffer
  int secret = 0x41414141; // adjacent variable

  // If cmd == 0x42 we write to buffer
  if (cmd == 0x42) {
    // Intentionally naive: no bounds check on n relative to buf size
    buf[n] = 0x00;
  }

  // Return secret so program behavior may subtly change if corrupted
  return secret;
}
```



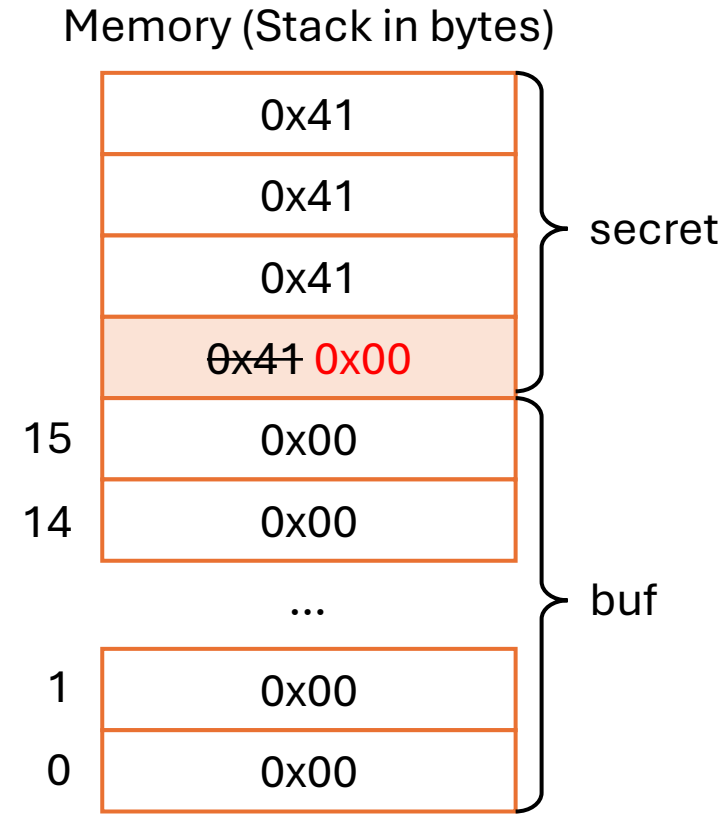
```c
int process_input(const uint8_t *data, size_t size) {
  if (size < 2) return 0; // need at least 2 bytes: cmd + length
  uint8_t cmd = data[0];
  uint8_t n = data[1]; // number of bytes to copy

  char buf[16]; // fixed-size buffer
  int secret = 0x41414141; // adjacent variable

  // If cmd == 0x42 we write to buffer
  if (cmd == 0x42) {
    // Intentionally naive: no bounds check on n relative to buf size
    buf[n] = 0x00;
  }

  // Return secret so program behavior may subtly change if corrupted
  return secret;
}
```

```
process_input({0x42, 16}, 2)
```



```c
int process_input(const uint8_t *data, size_t size) {
  if (size < 2) return 0; // need at least 2 bytes: cmd + length
  uint8_t cmd = data[0];
  uint8_t n = data[1]; // number of bytes to copy

  char buf[16]; // fixed-size buffer
  int secret = 0x41414141; // adjacent variable

  // If cmd == 0x42 we write to buffer
  if (cmd == 0x42) {
    // Intentionally naive: no bounds check on n relative to buf size
    buf[n] = 0x00;
  }

  // Return secret so program behavior may subtly change if corrupted
  return secret;
}
```

```
process_input({0x42, 16}, 2)
```

Issue:

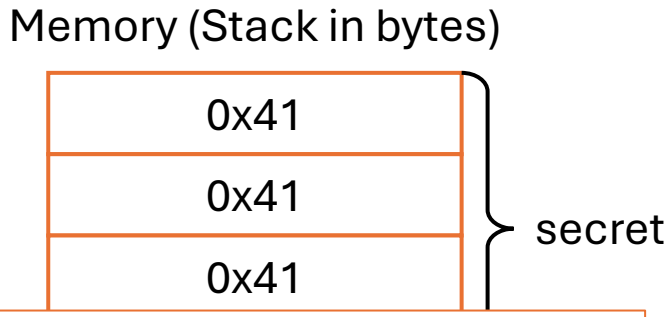
- There is an out-of-bounds buffer write!
- But the program will not crash!
- How do we still detect the bug?

Memory (Stack in bytes)



```c
int process_input(const uint8_t *data, size_t size) {
  if (size < 2) return 0; // need at least 2 bytes: cmd + length
  uint8_t cmd = data[0];
  uint8_t n = data[1]; // number of bytes to copy

  char buf[16]; // fixed-size buffer
```



2024 CWE Top 25 Most Dangerous Software Weaknesses

Top 25 Home | Share via: X | View in table format | Key Insights | Methodology

1 Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
CWE-79 | CVEs in KEV: 3 | Rank Last Year: 2 (up 1) ▲

2 Out-of-bounds Write
CWE-787 | CVEs in KEV: 18 | Rank Last Year: 1 (down 1) ▼

Issue:

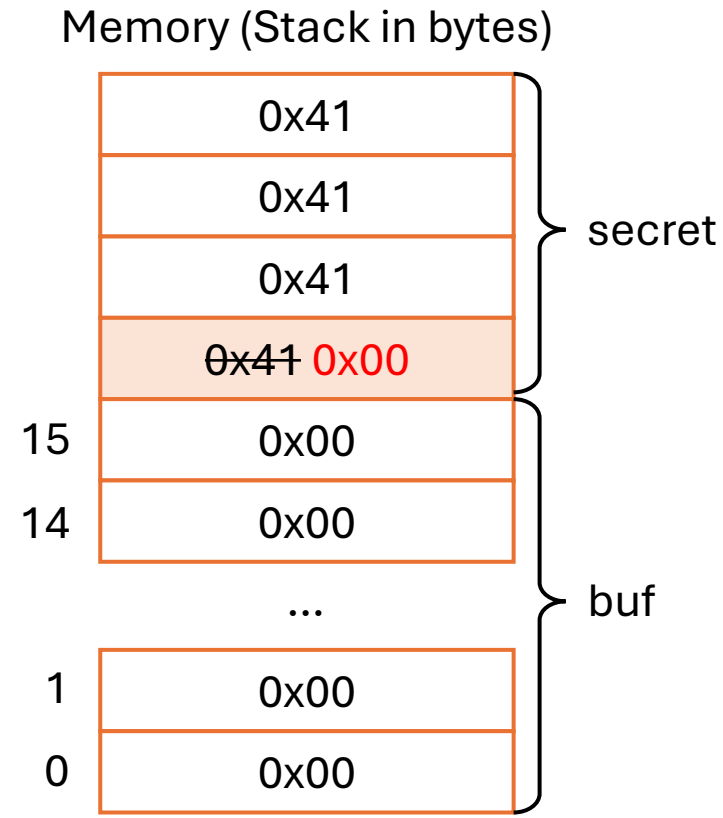
- There is an out-of-bounds buffer write!
- But the program will not crash!
- How do we still detect the bug?

```c
int process_input(const uint8_t *data, size_t size) {
  if (size < 2) return 0; // need at least 2 bytes: cmd + length
  uint8_t cmd = data[0];
  uint8_t n = data[1]; // number of bytes to copy

  char buf[16]; // fixed-size buffer
  int secret = 0x41414141; // adjacent variable

  // If cmd == 0x42 we write to buffer
  if (cmd == 0x42) {
    // Intentionally naive: no bounds check on n relative to buf size
    buf[n] = 0x00;
  }

  // Return secret so program behavior may subtly change if corrupted
  return secret;
}
```




```
process_input({0x42, 16}, 2)
```

Issue:

- There is an out-of-bounds buffer write!
- But the program will not crash!
- How do we still detect the bug?

Solution:

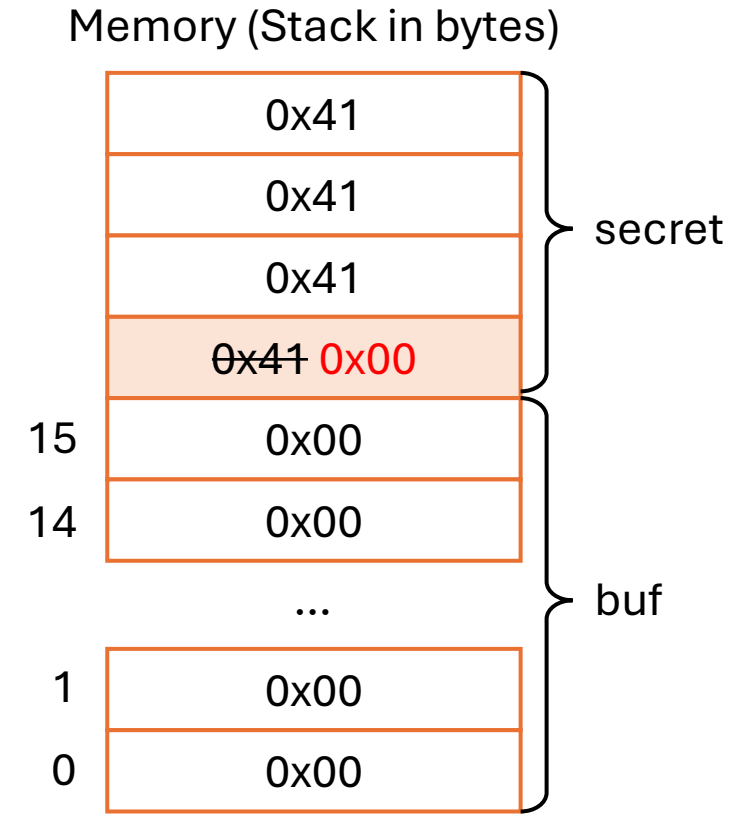
- Add Sanitizer!

```c
int process_input(const uint8_t *data, size_t size) {
  if (size < 2) return 0; // need at least 2 bytes: cmd + length
  uint8_t cmd = data[0];
  uint8_t n = data[1]; // number of bytes to copy

  char buf[16]; // fixed-size buffer
  int secret = 0x41414141; // adjacent variable

  // If cmd == 0x42 we write to buffer
  if (cmd == 0x42) {
    if (n >= 16) SANITIZER_ERROR("out-of-bounds write")
    buf[n] = 0x00;
  }

  // Return secret so program behavior may subtly change if corrupted
  return secret;
}
```




```
process_input({0x42, 16}, 2)
```

Issue:

- There is an out-of-bounds buffer write!
- But the program will not crash!
- How do we still detect the bug?

Solution:

- Add Sanitizer!

Program Instrumentation

P

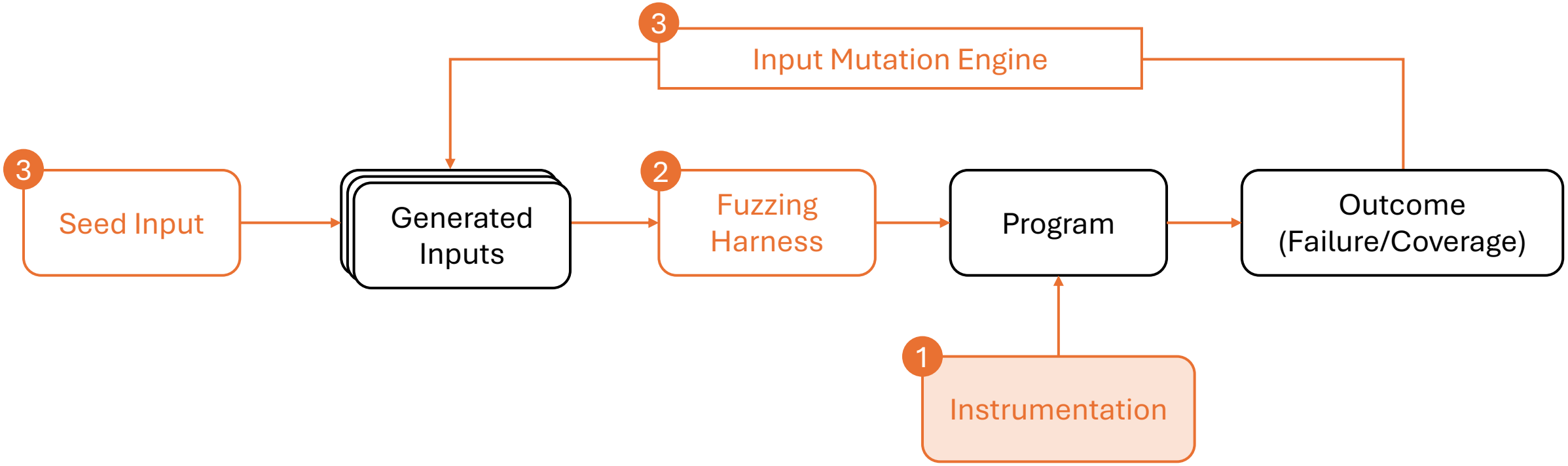
Original Program

P'

Instrumented Program

P and P' are equivalent under normal behavior
P' can expose more failure modes than P

Fuzzing: The Loop



Fuzzing Harness

- For unit tests:
  - We come up with a single input-output pair
  - We write a test case that runs the program on input, compares the output
- For fuzzing:
  - We have automatically generated input
  - We write a test case that runs the program on input // ← HARNESS

Fuzzing Harness

- For unit tests:
  - We come up with a single input-output pair
  - We write a test case that runs the program on input, compares the output
- For fuzzing:
  - We have automatically generated input
  - We write a test case that runs the program on input // ← HARNESS

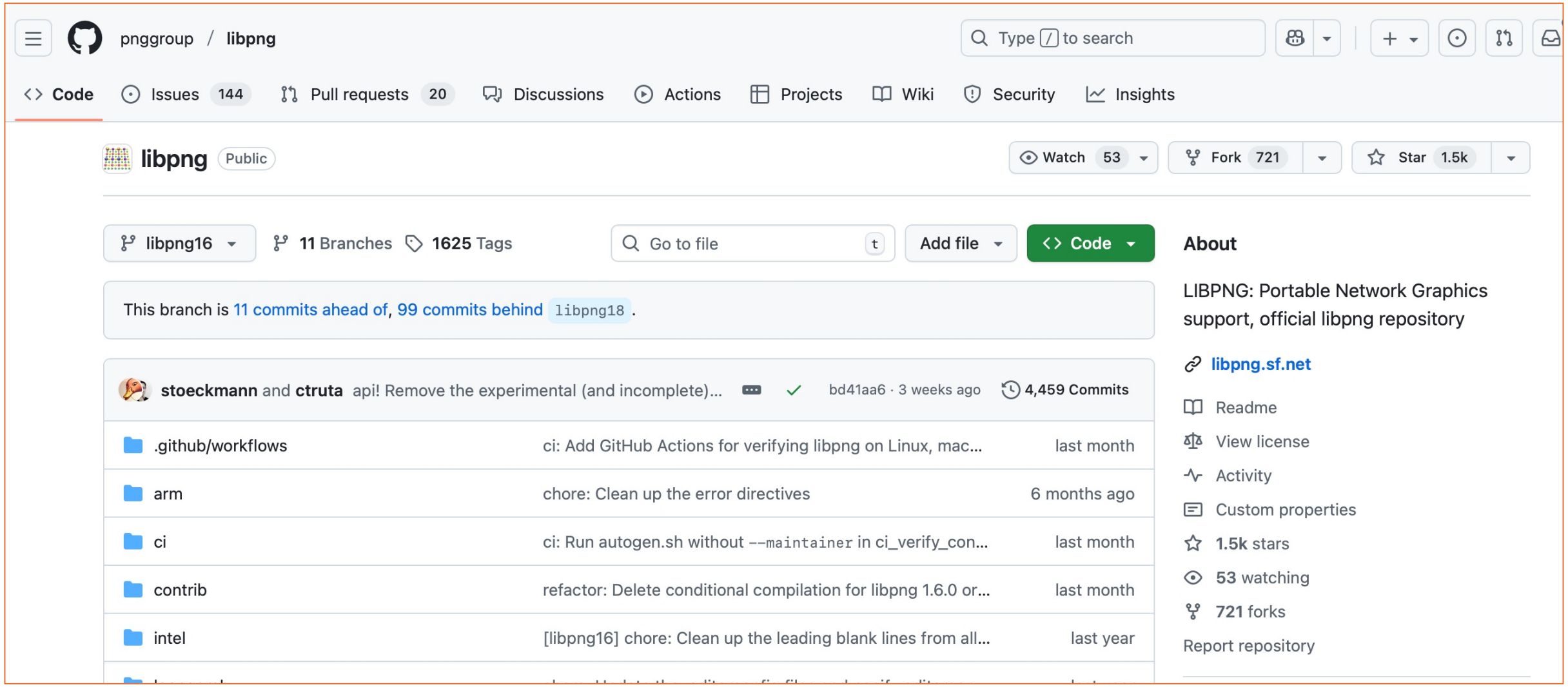
We have to do this for a lot of times!

Fuzzing Harness

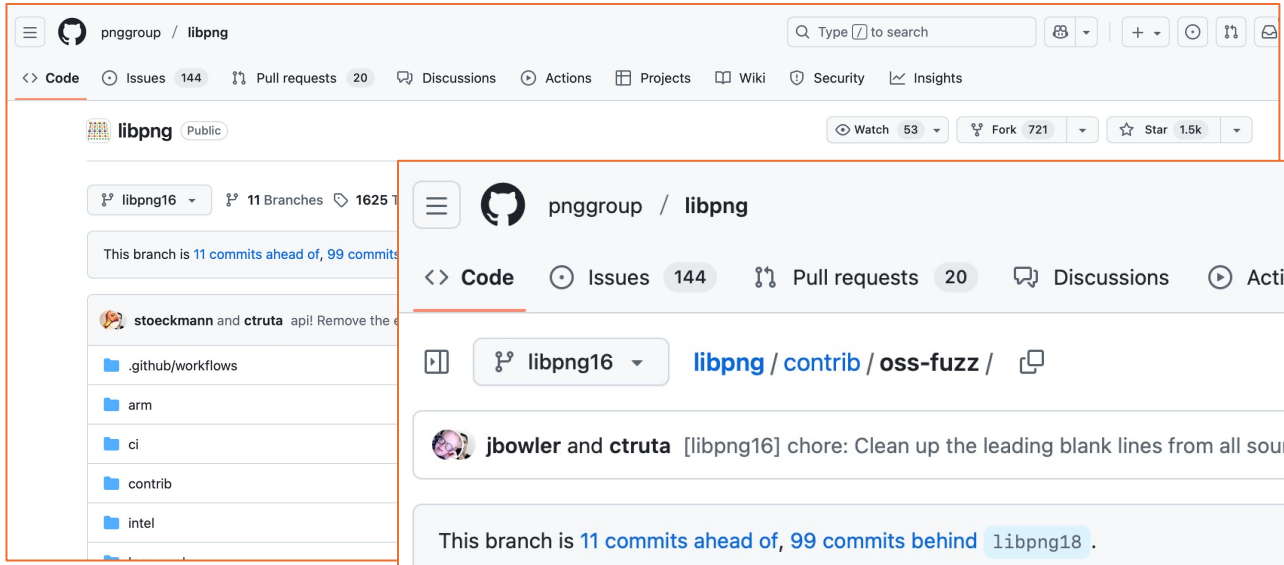
- For unit tests:
  - We come up with a single input-output pair
  - We write a test case that runs the program on input, compares the output
- For fuzzing:
  - We have automatically generated input
  - We write a test case that runs the program on input // ← **HARNESS**

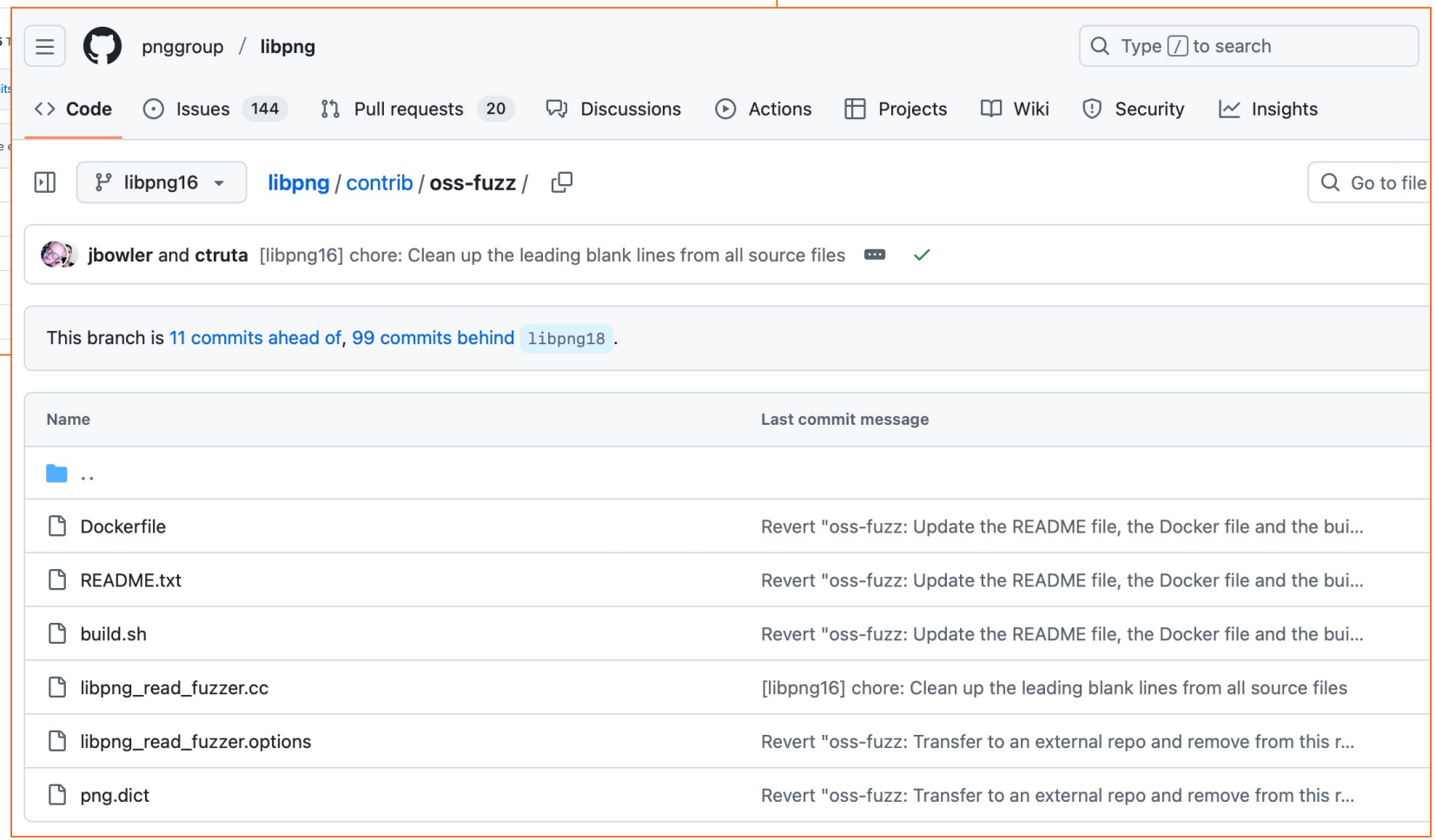
We have to do this for a lot of times!

Can we just do this one time or only a few times?

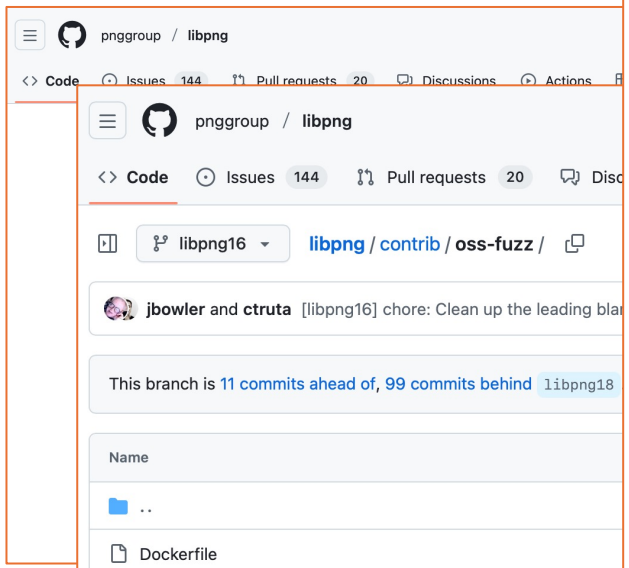


https://github.com/pnggroup/libpng





https://github.com/pnggroup/libpng



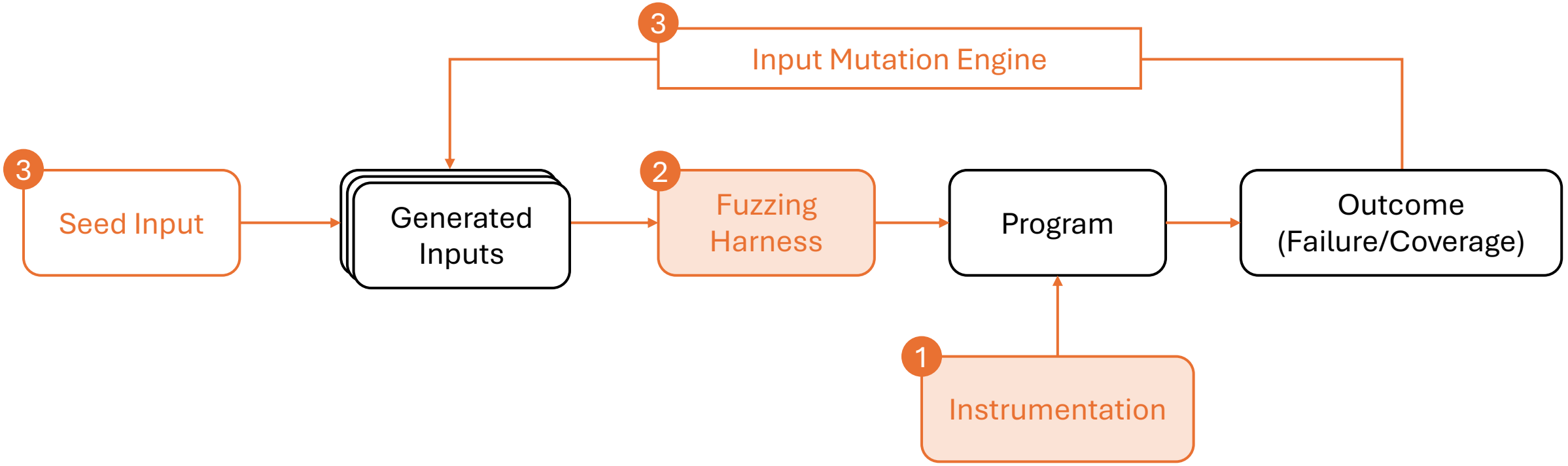

```
97  // Entry point for LibFuzzer.
98  // Roughly follows the libpng book example:
99  // http://www.libpng.org/pub/png/book/chapter13.html
100 extern "C" int LLVMFuzzerTestOneInput(const uint8_t* data, size_t size) {
101   if (size < kPngHeaderSize) {
102     return 0;
103   }
104
105   std::vector<unsigned char> v(data, data + size);
106   if (png_sig_cmp(v.data(), 0, kPngHeaderSize)) {
107     // not a PNG.
108     return 0;
109   }
110
111   PngObjectHandler png_handler;
112   png_handler.png_ptr = nullptr;
113   png_handler.row_ptr = nullptr;
114   png_handler.info_ptr = nullptr;
115   png_handler.end_info_ptr = nullptr;
116
117   png_handler.png_ptr = png_create_read_struct
118     (PNG_LIBPNG_VER_STRING, nullptr, nullptr, nullptr);
119   if (!png_handler.png_ptr) {
120     return 0;
121   }
122
123   png_handler.info_ptr = png_create_info_struct(png_handler.png_ptr);
124   if (!png_handler.info_ptr) {
125     PNG_CLEANUP
126     return 0;
127   }
128
129   png_handler.end_info_ptr = png_create_info_struct(png_handler.png_ptr);
130   if (!png_handler.end_info_ptr) {
131     PNG_CLEANUP
132     return 0;
133   }
```

https://github.com/pnggroup/libpng/blob/libpng16/contrib/oss-fuzz/libpng_read_fuzzer.cc

Fuzzing Harness: Key Message

- A single wide-capturing test case that takes a single buffer as input
  - Reject bad inputs that we do not care about in the beginning
  - Simulate complete usage of a library or a package
  - Trigger sanitization errors or capture failures and exceptions
- In practice, a library may be accompanied multiple harnesses
  - But not to the level of the amount of unit test cases

Fuzzing: The Loop



Seed Input and Mutation Engine

- Seed Input
  - Typically some valid inputs to a program
  - E.g., an HTML for a browser, a PNG for libpng, a JSON for a JSON parser
- Mutation Engine
  - With a single input, we may get feedback from the execution:
    - E.g., coverage, error, command line outputs, etc.
  - We may use the coverage to guide the mutation engine to change the seed input to something else
    - E.g., JSON "`[1, 2, 3]`" → "`[1, 2,]`"
  - Hopefully, the mutation leads us to a better coverage

Infinite Monkey Theorem

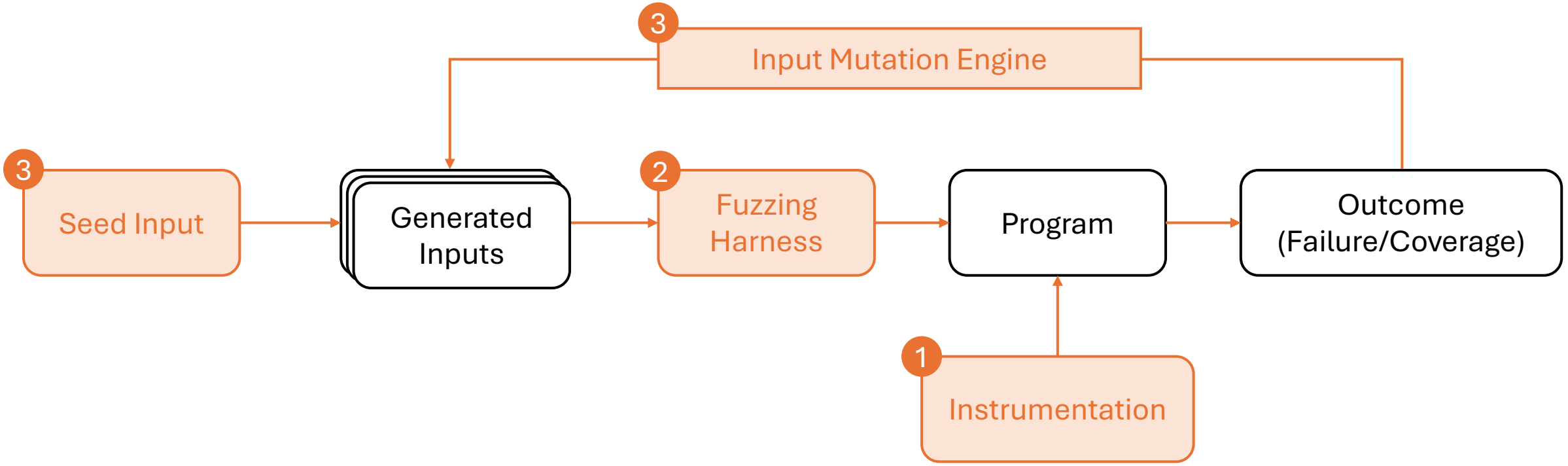
“a monkey hitting keys independently and at random on a typewriter keyboard for an infinite amount of time will almost surely type any given text, including the complete works of William Shakespeare.”



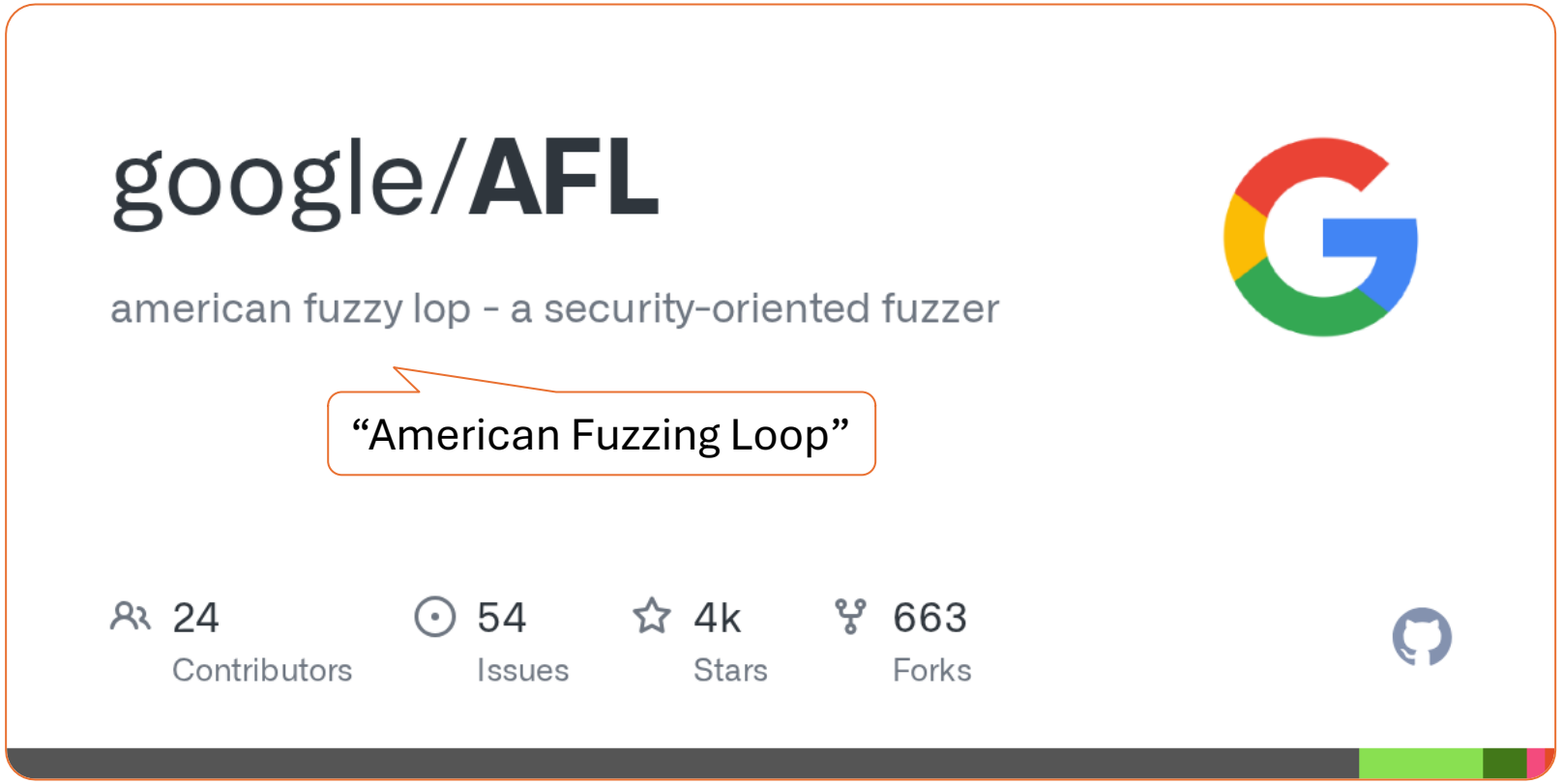
Seed Input and Mutation Engine

- Seed Input
  - Typically some valid inputs to a program
  - E.g., an HTML for a browser, a PNG for libpng, a JSON for a JSON parser
- Mutation Engine
  - With a single input, we may get feedback from the execution:
    - E.g., coverage, error, command line outputs, etc.
  - We may use the coverage to guide the mutation engine to change the seed input to something else
    - E.g., JSON "`[1, 2, 3]`" → "`[1, 2,]`"
  - Hopefully, the mutation leads us to a better coverage

Fuzzing: The Loop



Fuzzing in the Wild: AFL



American Fuzzy Lop (AFL), Michał Zalewski (lcamtuf), 2013

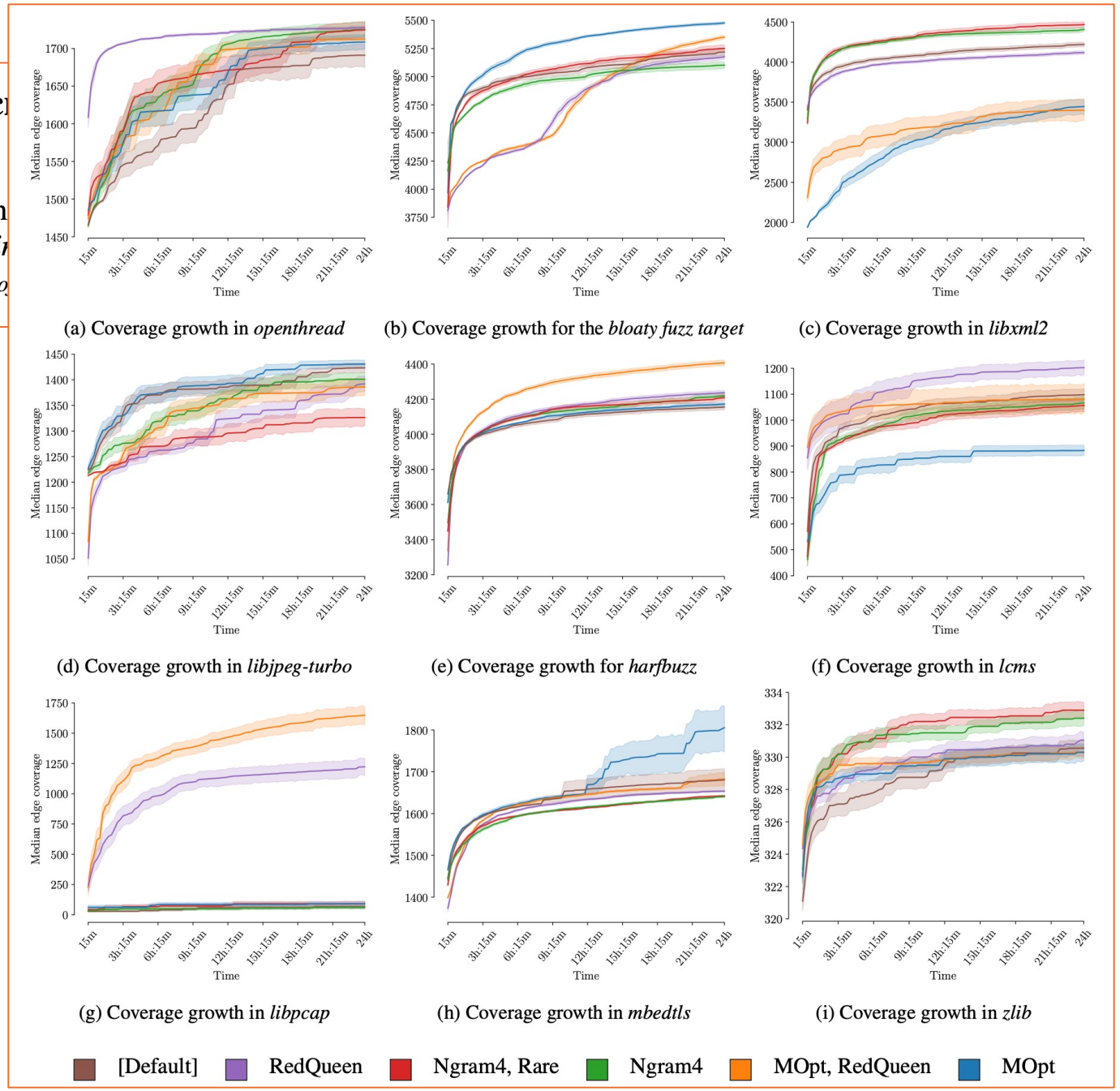
AFL++: Combining Incremental Steps of Fuzzing Research

Andrea Fioraldi†, Dominik Maier‡, Heiko Eißfeldt, Marc Heuse§
{andrea, dominik, heiko, marc}@aflplus.plus
†*Sapienza University of Rome*, ‡*TU Berlin*, §*The Hacker's Choice*

American Fuzzy Lop (AFL), Michał Zalewski (lcamtuf), 2013

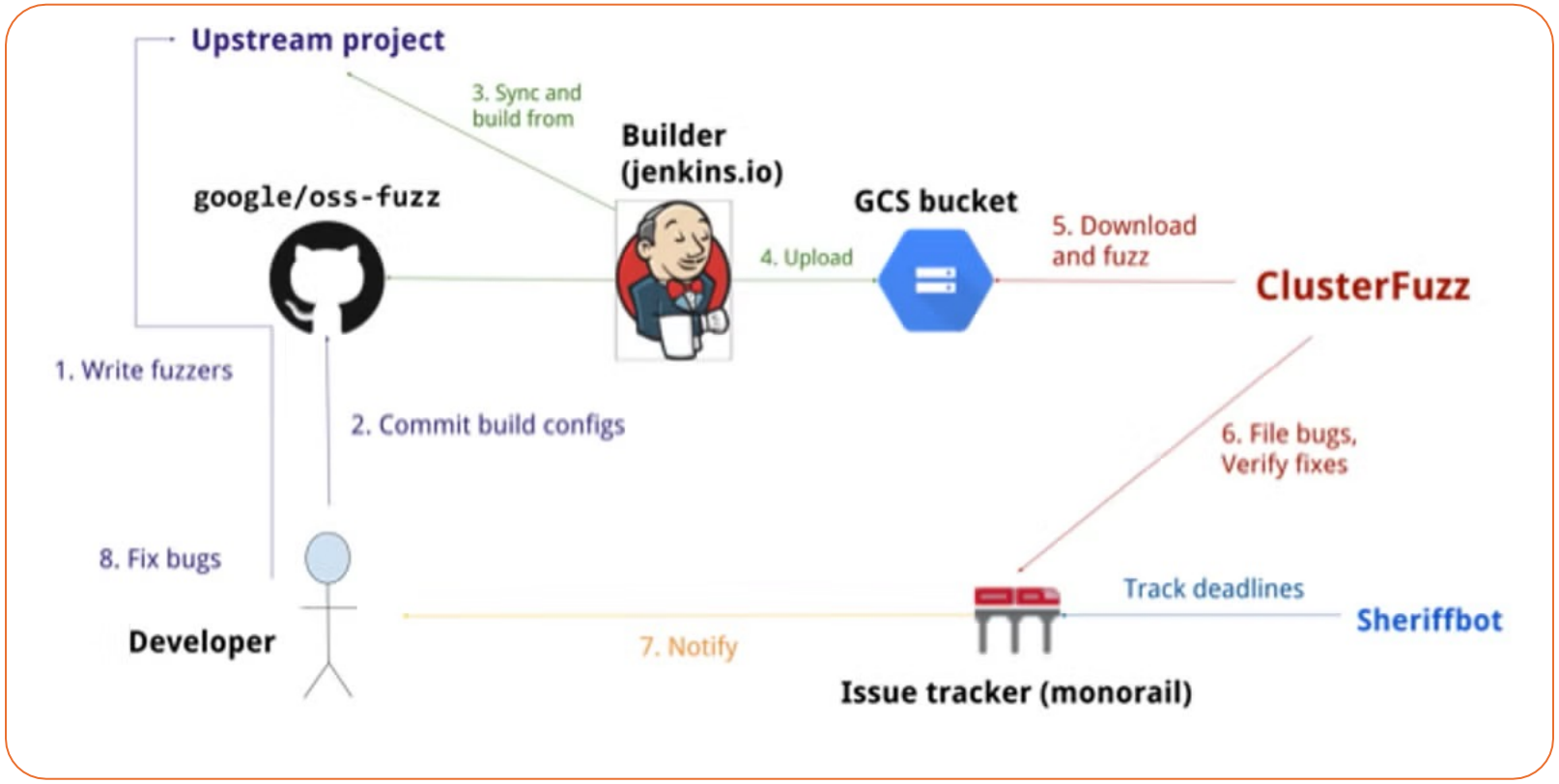
AFL++: Combining Incr

Andrea Fioraldi†, Domin
{andrea, domir
†Sapienza University o



American Fuzzy Lop (AFL), Michał Zalewski (lcamtuf), 2013

Fuzzing in the Wild: OSS-Fuzz



OSS-Fuzz: Google's Continuous Fuzzing Service for Open Source Software, Serebryany, USENIX Security 2017

Fuzzing in the Wild: OSS-Fuzz

Coverage Report

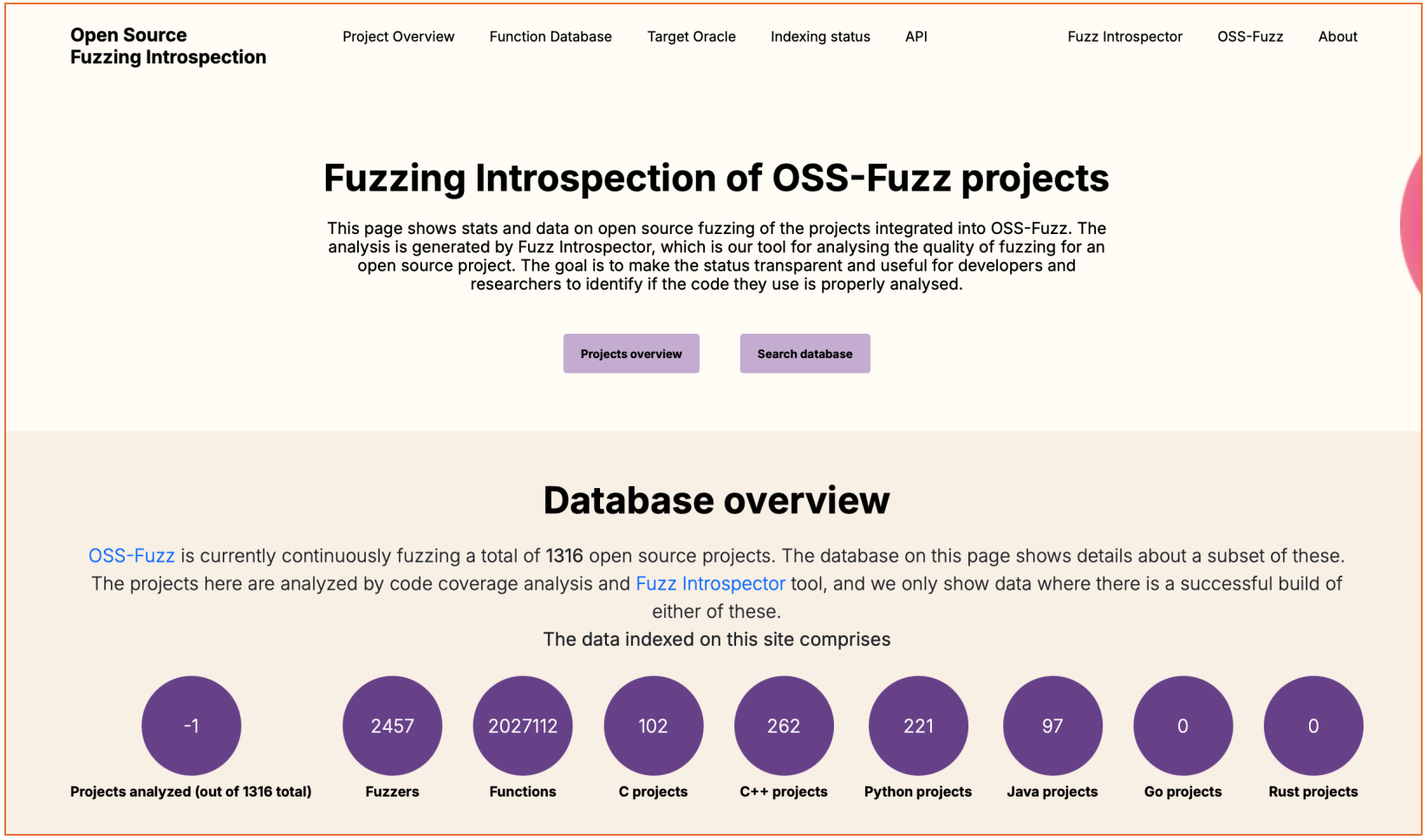
View results by: Directories | Files

PATH	LINE COVERAGE	FUNCTION COVERAGE	REGION COVERAGE
autofit/	89.65% (6801/7586)	88.64% (78/88)	86.09% (3280/3810)
base/	73.47% (7650/10413)	77.81% (249/320)	62.89% (5010/7966)
bdf/	94.60% (2503/2646)	97.30% (36/37)	92.54% (2133/2305)
cff/	91.68% (5048/5506)	92.08% (93/101)	89.75% (2880/3209)
cid/	96.03% (1717/1788)	93.94% (31/33)	94.75% (1100/1161)
gzip/	95.54% (1435/1502)	94.12% (32/34)	88.73% (1299/1464)
lzw/	79.80% (482/604)	83.33% (15/18)	76.90% (263/342)
pcf/	94.46% (1722/1823)	83.87% (26/31)	93.17% (1445/1551)
pfr/	91.69% (2603/2839)	91.49% (43/47)	92.17% (2049/2223)
psaux/	91.19% (11504/12615)	96.12% (198/206)	88.69% (5990/6754)
pshinter/	95.20% (2957/3106)	92.41% (73/79)	91.25% (1356/1486)
psnames/	99.36% (466/469)	100.00% (11/11)	98.70% (227/230)
raster/	96.07% (2002/2084)	90.48% (38/42)	91.34% (1539/1685)
sfnt/	89.03% (8089/9086)	96.00% (144/150)	87.35% (6187/7083)
smooth/	90.62% (899/992)	82.14% (23/28)	85.52% (685/801)
truetype/	95.29% (12299/12907)	99.63% (269/270)	89.28% (7409/8299)
type1/	93.10% (4154/4462)	93.94% (62/66)	90.94% (2811/3091)
type42/	95.79% (1662/1735)	90.62% (29/32)	94.50% (1099/1163)
winfonts/	96.16% (826/859)	100.00% (13/13)	94.33% (632/670)
TOTALS	90.12% (74819/83022)	91.10% (1463/1606)	85.71% (47394/55293)

```
59          /* multiply a given value by the CORDIC shrink factor */
60          static FT_Fixed
61          ft_trig_downscale( FT_Fixed  val )
62    111k  {
63    111k    FT_Int  s = 1;
64    111k
65    111k
66    111k    if ( val < 0 )
67      0     {
68      0       val = -val;
69      0       s = -1;
70      0     }
71    111k
72    111k    /* 0x40000000 comes from regression analysis between true */
73    111k    /* and CORDIC hypotenuse, so it minimizes the error       */
74    111k    val = (FT_Fixed)(
75    111k            ( (FT_UInt64)val * FT_TRIG_SCALE + 0x40000000UL ) >> 32 );
76    111k
77    111k    return s < 0 ? -val : val;
78    111k  }
```

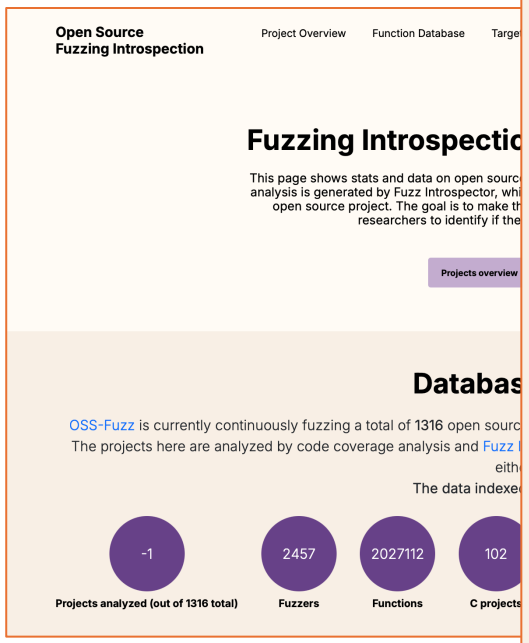
OSS-Fuzz: Google's Continuous Fuzzing Service for Open Source Software, Serebryany, USENIX Security 2017

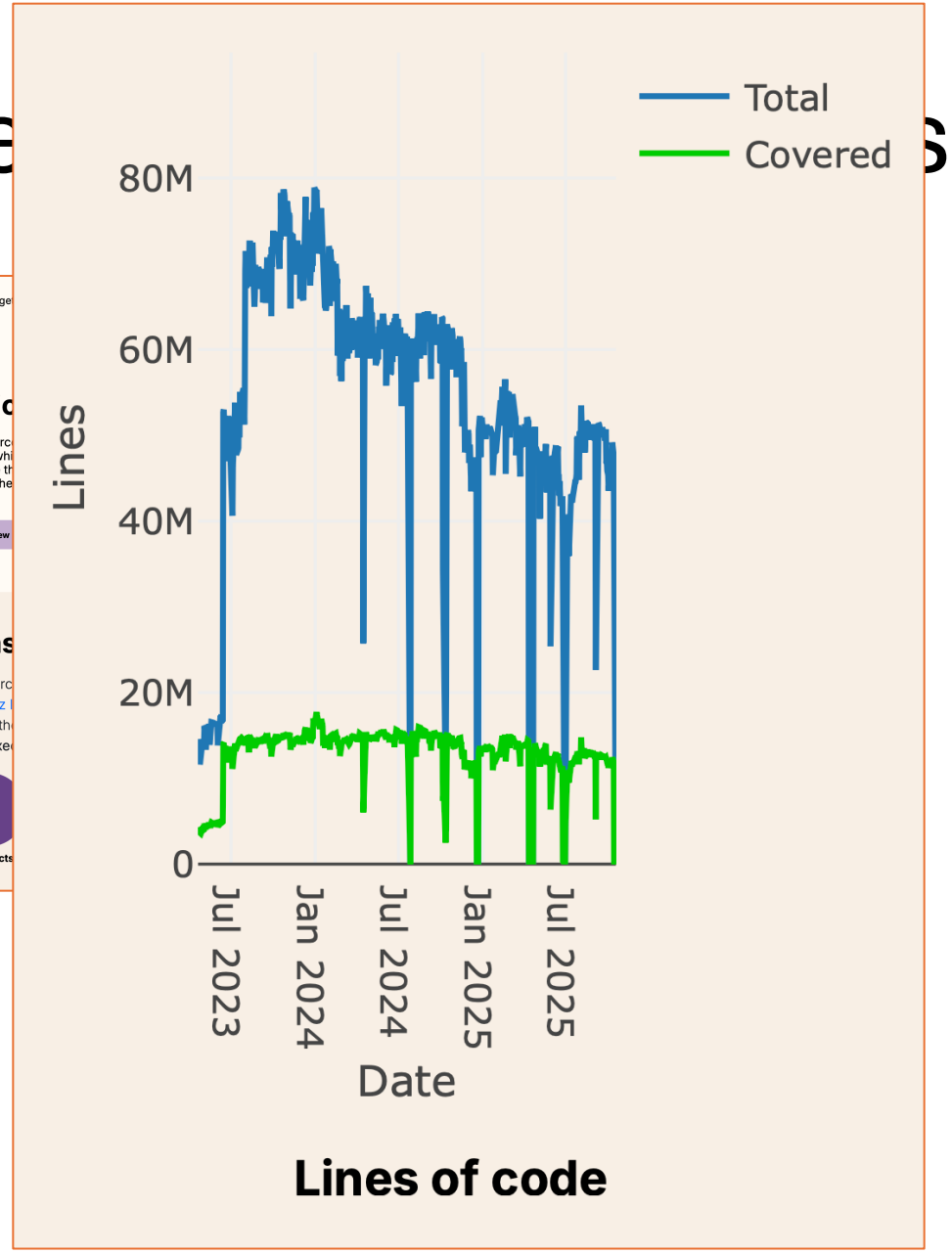
Fuzzing in the Wild: Fuzzing Introspection



Fuzz Introspector, Ada Logics & OSS-Fuzz, 2023 — github.com/ossf/fuzz-introspector

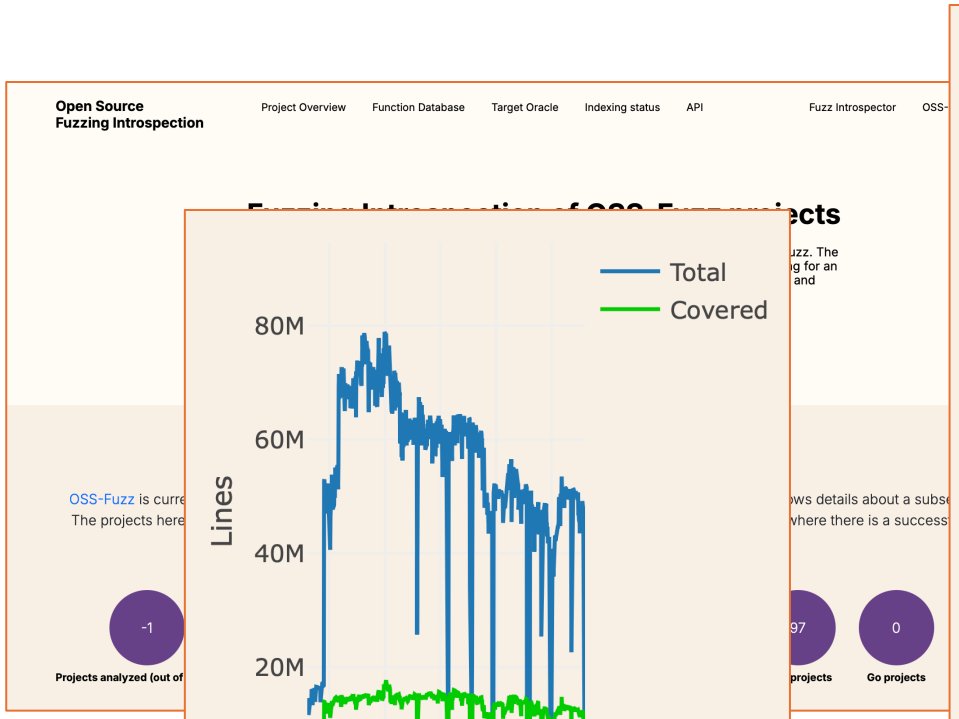
Fuzzing in the spection

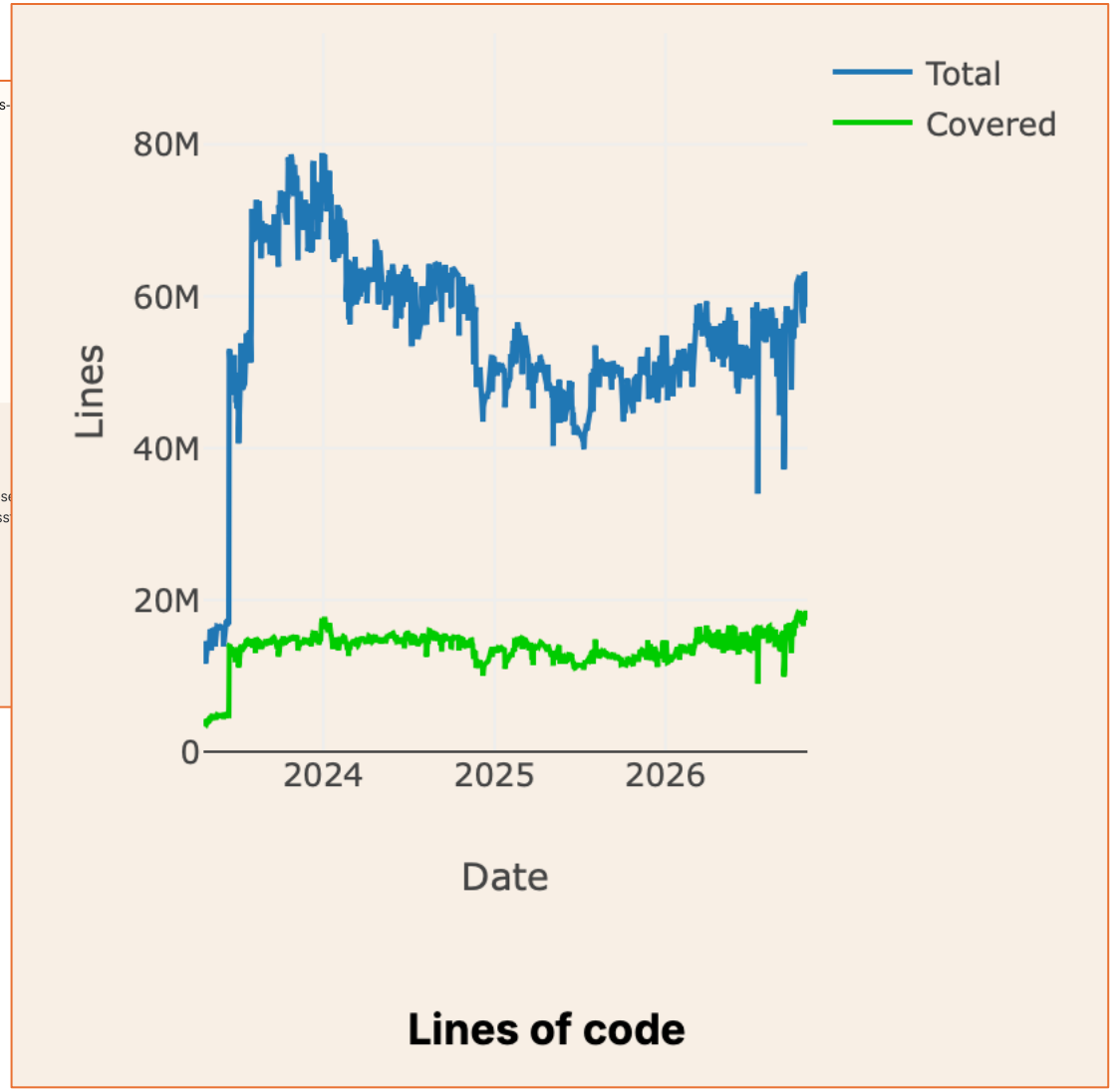




Fuzz Introspector, Ada Logics & OSS-Fuzz, 2023 — github.com/ossf/fuzz-introspector

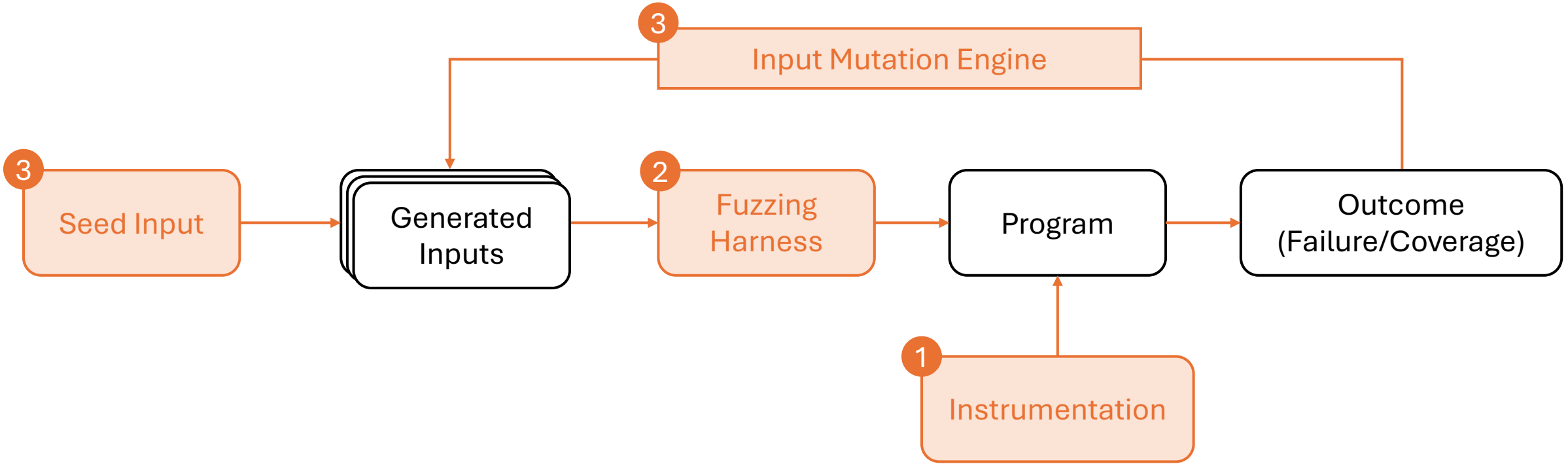
Fuzzing in the Wild: Fuzzing Introspection





Fuzz Introspector, Ada Logics & OSS-Fuzz, 2023 — github.com/ossf/fuzz-introspector

How Can LLM Help?



How Can LLM Help?

- LLM for instrumentation:
  - Capture more vulnerability patterns than traditional sanitizers
  - E.g., Path Traversal, Reading from external logs, etc.
- LLM for fuzzing harness generation:
  - Capture more library usage pattern
  - E.g., For libpng, simulate browser usage, PDF reader usage, etc.
- LLM for seed input generation & mutation:
  - Similar to unit test generation, but can accept more types of feedback
  - E.g., Looking at uncovered lines, mutate input towards those lines

How Effective Are They? Exploring Large Language Model Based Fuzz Driver Generation

Cen Zhang
Nanyang Technological University
Singapore

Yaowen Zheng*
Nanyang Technological University
Singapore

Mingqiang Bai
IIE, CAS; Sch of Cyber Security, UCAS
Beijing, China

Yeting Li
IIE, CAS; Sch of Cyber Security, UCAS
Beijing, China

Wei Ma
Nanyang Technological University
Singapore

Xiaofei Xie
Singapore Management University
Singapore

Yuekang Li
The University of New South Wales
Sydney, Australia

Limin Sun
IIE, CAS; Sch of Cyber Security, UCAS
Beijing, China

Yang Liu
Nanyang Technological University
Singapore

How Effective Are They? Exploring Large Language Model Based

Cen Z
Nanyang Technol
Singa

Yetin
IIE, CAS; Sch of Cyb
Beijing,

Yueka
The University of N
Sydney, A

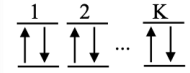
Legend: ☐ Basic API info only ☐ Contain extended API info [Content] Target API specific content in prompt template

Prompt Strategy Design

Design I - 『Query With Different Types of API Info』

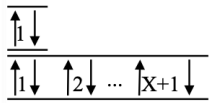
- **Basic Info**: Precisely specified & generally accessible, *e.g.*, header file and API declaration
- **Extended Info**: Not guaranteed in quality and availability, *e.g.*, API documentation and usage snippets

Design II - 『Query Repeatedly』



- Repeat whole query process K times for K independent results
- Suffix "-K" in name

Design III - 『Query Iteratively』



- **Non-Iterative**: one generation query
- **Iterative**: one generation query with up to X fix queries

Acronym	Prompts (Generation + Fix)	EX	IT
NAIVE-K	①	✗	✗
BACTX-K	① + ②	✗	✗
DOCTX-K	①+②+④	✓	✗
UGCTX-K	①+②+③	✓	✗
BA-ITER-K	① + ② + ⓐ	✗	✓
ALL-ITER-K	① + ② or ①+②+③ or ①+②+④ + ⓐ or ⓐ + ⓑ	✓	✓

EX→use extended API info; IT→iterative query & fix

Generation Prompt Template

① **Task description**

② **[Header file inclusion]**

③ **[Example code snippets which shows API usage]**

④ **[API documentation]**

② **[API declaration]**

① **Task description**

Generation Prompt Example

```
// The following is a fuzz driver written in C language, complete
the implementation. Output the continued code in reply only.

#include "bpf/libbpf.h"

// @ examples of API usage from bpf-loader.c
// void test_bpf(const char *bpf_file) {
//    ...
//    obj = bpf_open_mem(_buffer, _size, NULL);
//    ... }

/* @brief it creates a bpf_object by reading the BPF objects ...
 * @param buf pointer to the buffer containing BPF ... */
extern bpf* bpf_open_mem(char *buf, int sz, struct opts *opts);

// the following function fuzzes bpf_open_mem
int LLVMFuzzerTestOneInput(const uint8_t *Data, size_t Size) {
```

Fix Prompt Template

ⓐ ```` [Code of error fuzz driver] ```` [One sentence error summary]

ⓑ [Error line code] [Error details] [Other supplemental info]

ⓐ **Task description**

Fix Prompt Example

```
```
... obj = bpf_open_mem(Data, Size, NULL); ...
```
The above C code can be built successfully but will crash immediately during execution(ASAN-assertion failure)

Error line: `obj = bpf_open_mem(Data, Size, NULL);`
Crash stack and nearby code:
#3 0x5f7744 in bpf_open_mem /src/bpf/libbpf.c:256:28

 256 assert(buf != NULL && sz != 0);

Based on the above information, fix the code.
```

Figure 2: Prompt Strategies Overview. K is 1 or 40 in our evaluation and X is 5. The examples are simplified for demonstration purpose. In the fix prompt example, the driver error is caused by missing check of Size > 0 before calling the API, and the nearby code of #3 stack frame hints the error.

How Effective Are They? Exploring Large Language Model Based Fuzz Driver Generation

Nan

IIE, C

The

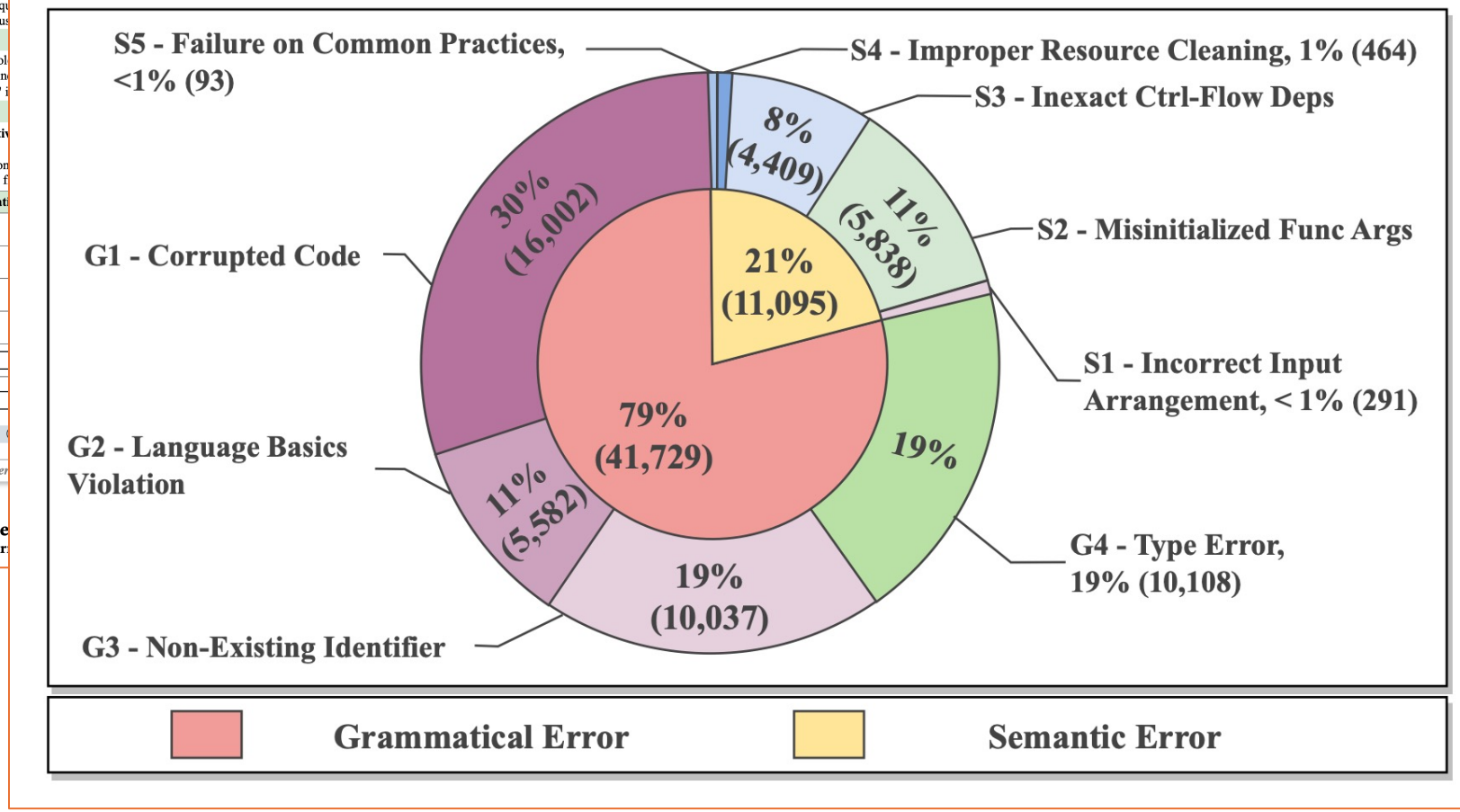
Legend: Basic API info only | Contain extended API info | [Content] Target API specific content in prompt template

Prompt Strategy Design	Generation Prompt Template	Generation Prompt Example
Design I -『Query With Different Types of API Info』 - **Basic Info**: Precisely specified & ge *e.g.*, header file and API declaration - **Extended Info**: Not guaranteed in qu ability, *e.g.*, API documentation and us	① `Task description`	`// The following is a fuzz driver written in C language, complete`
Design II -『Query Repeatedly』 1 2 ... K - Repeat whol times for K in - Suffix "-K" i		
Design III -『Query Iteratively』 1 - **Non-Iterati** query 1 2 ... X+1 - **Iterative**: on with up to X f		

Acronym	Prompts (Generati
NAIVE-K	①
BACTX-K	① + ②
DOCTX-K	①+②+④
UGCTX-K	①+②+③
BA-ITER-K	① + ② +
ALL-ITER-K	① + ② or ①+②+③ + or ①+②+④

EX→use extended API info; IT→iter

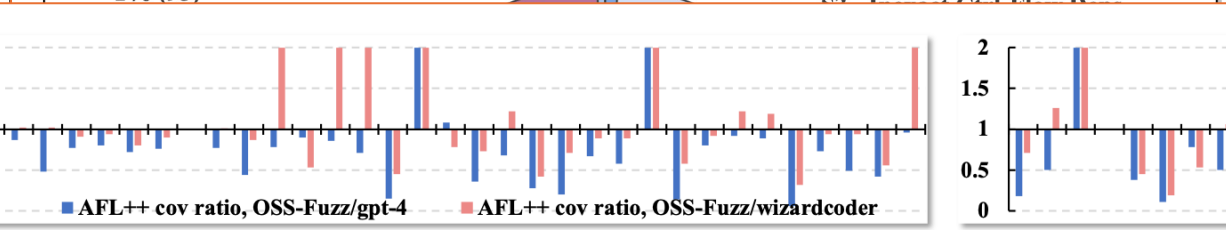
Figure 2: Prompt Strate
prompt example, the driver er

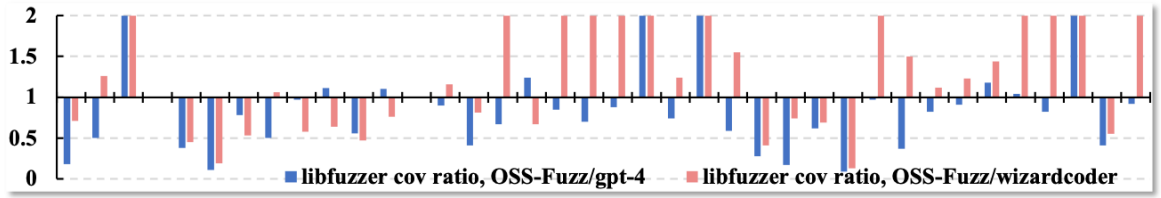


How Effective Are They? Exploring Large Language Model Based Fuzz Driver Generation

Legend: Basic API info only | Contain extended API info | [Content] Target API specific content in prompt template

Prompt Strategy Design	Generation Prompt Template	Generation Prompt Example
Design I - 『Query With Different Types of API Info』	① Task description	// The following is a fuzz driver written in C language, complete
- Basic Info: Prec... e.g., header file a...		
- Extended Info: ... ability, e.g., API ...		
Design II - 『Qu...		





(a) Comparison in Average Coverage Ratio. Ratio = OSS-Fuzz/LLM, ratio < 1 → OSS-Fuzz's driver has lower coverage.





(b) Comparison in the Difference of Average Unique Crashes. Diff = OSS-Fuzz - LLM, diff < 0 → OSS-Fuzz's driver find less unique crashes.

Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models

Yinlin Deng
University of Illinois
Urbana-Champaign, USA
yinlind2@illinois.edu

Chunqiu Steven Xia
University of Illinois
Urbana-Champaign, USA
chunqiu2@illinois.edu

Haoran Peng
University of Science and
Technology of China, China
hurrypeng@mail.ustc.edu.cn

Chenyuan Yang
University of Illinois
Urbana-Champaign, USA
cy54@illinois.edu

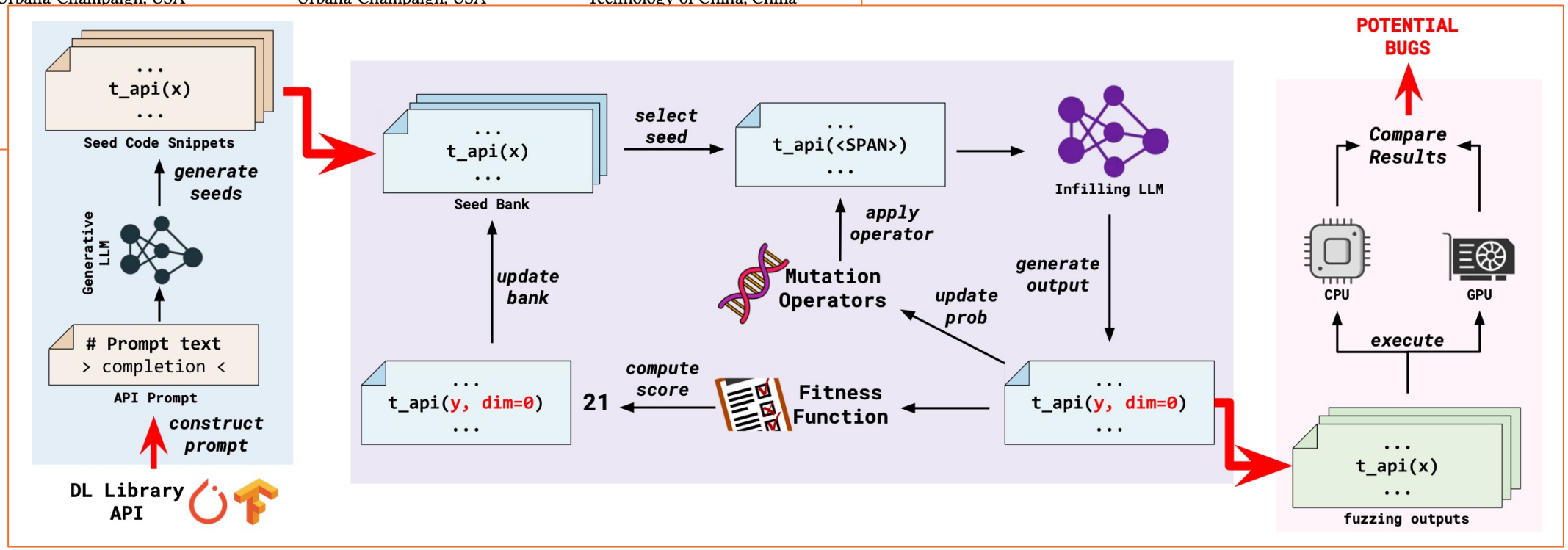
Lingming Zhang
University of Illinois
Urbana-Champaign, USA
lingming@illinois.edu

Large Language Models Are Zero-Shot Fuzzers: Fuzzing Deep-Learning Libraries via Large Language Models

Yinlin Deng
University of Illinois Urbana-Champaign, USA

Chunqiu Steven Xia
University of Illinois Urbana-Champaign, USA

Haoran Peng
University of Science and Technology of China, China



Large Language Models Are Zero-Shot Fuzzers:

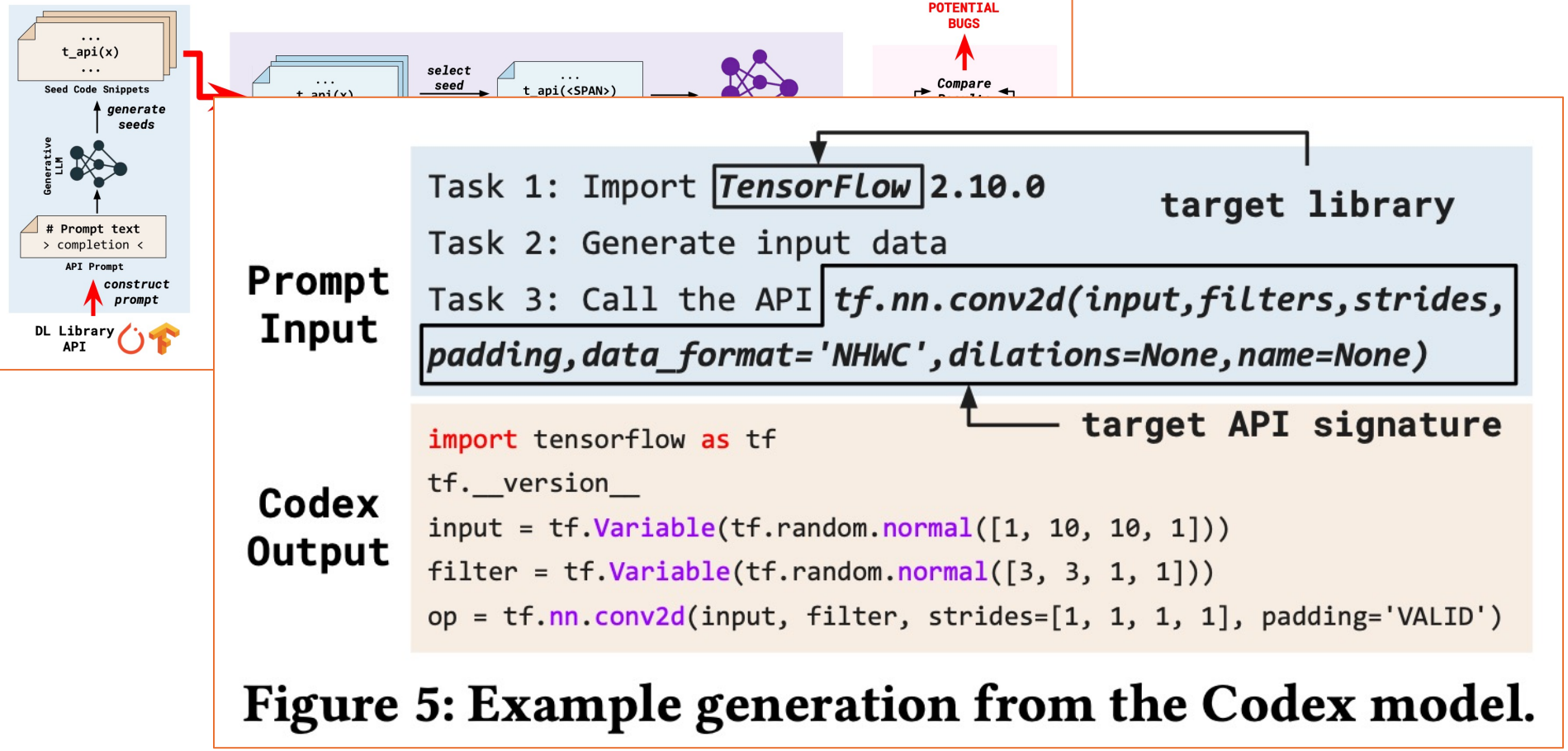


Figure 5: Example generation from the Codex model.

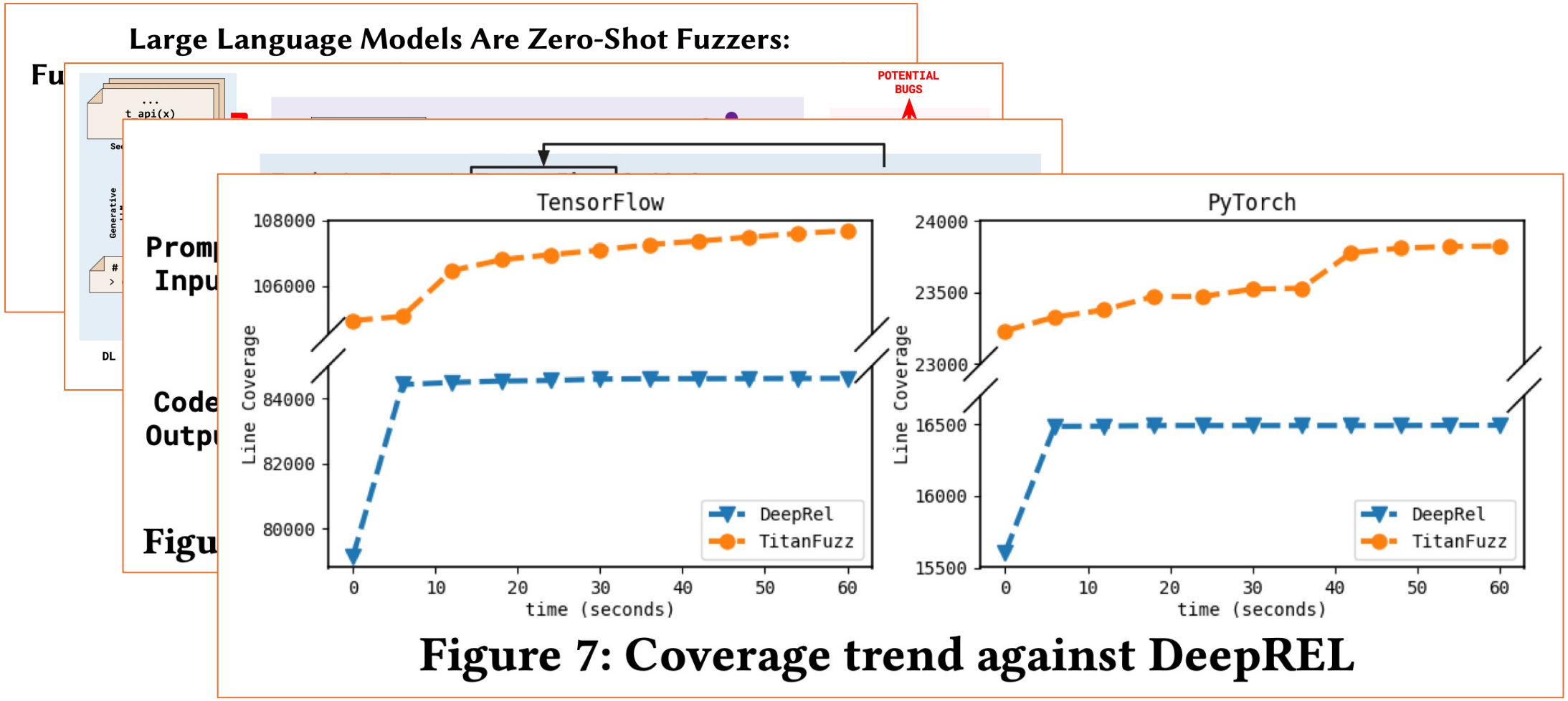
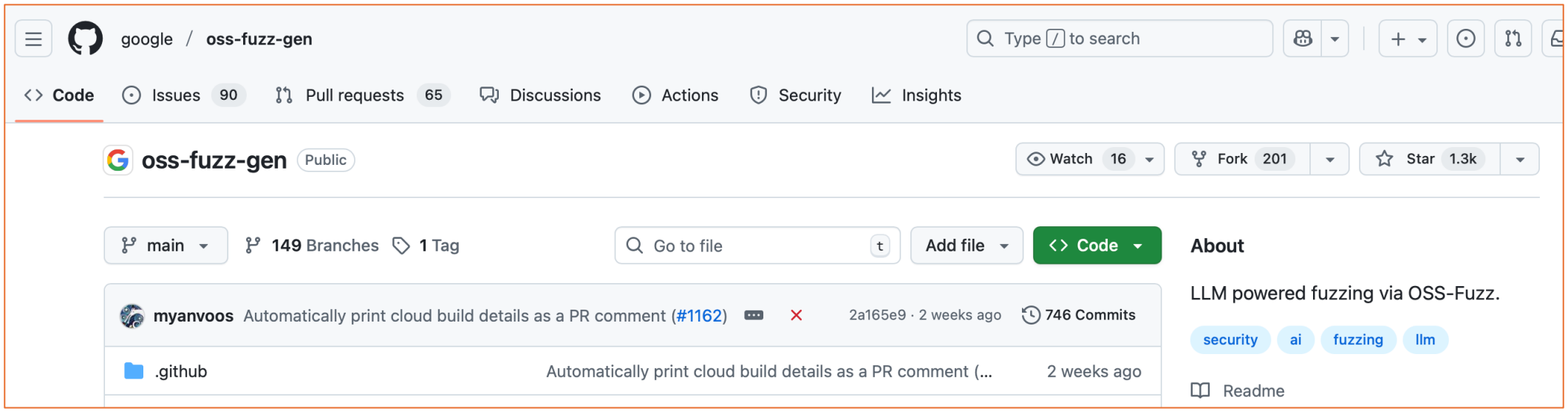
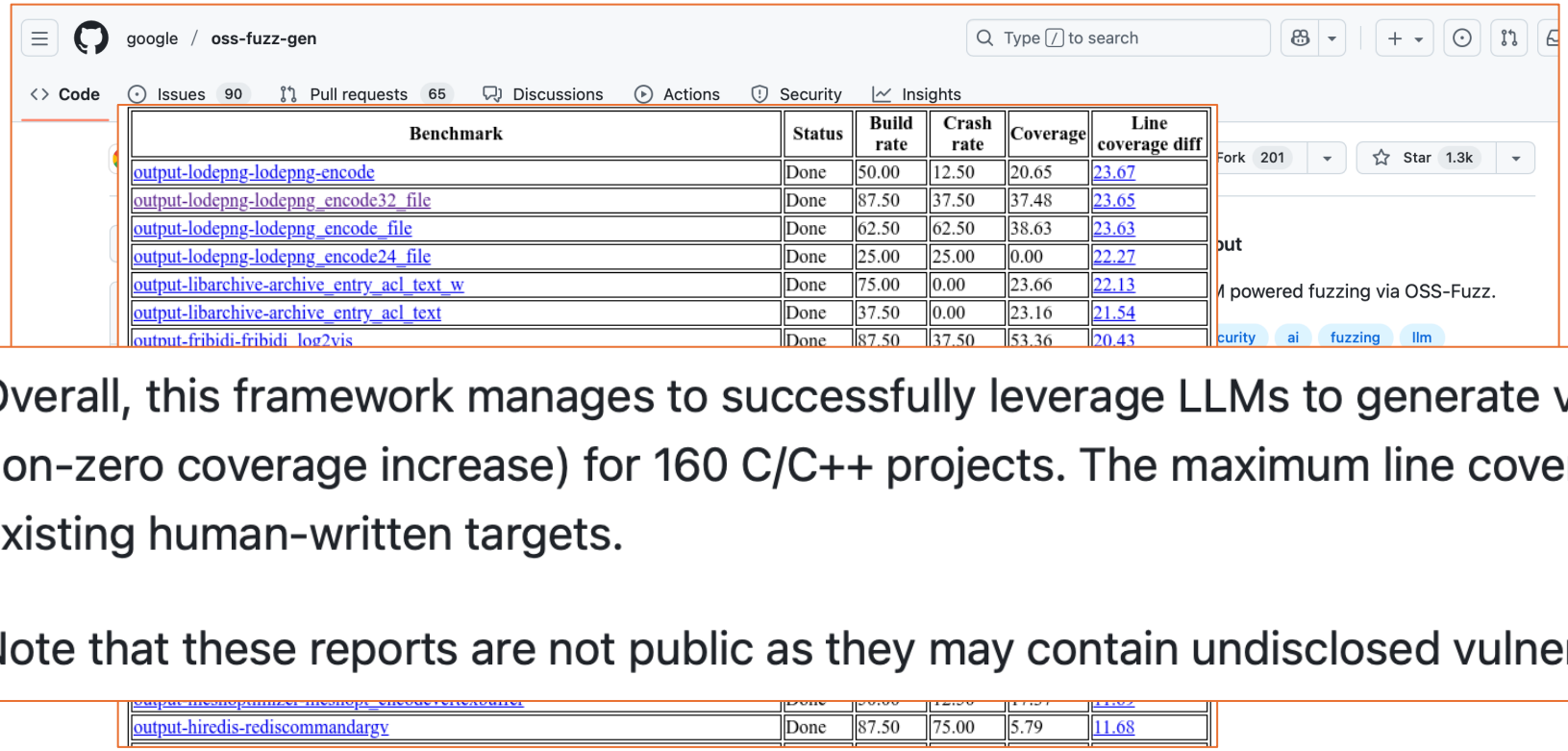


Figure 7: Coverage trend against DeepREL



https://github.com/google/oss-fuzz-gen



Benchmark	Status	Build rate	Crash rate	Coverage	Line coverage diff
output-lodepng-lodepng-encode	Done	50.00	12.50	20.65	23.67
output-lodepng-lodepng_encode32_file	Done	87.50	37.50	37.48	23.65
output-lodepng-lodepng_encode_file	Done	62.50	62.50	38.63	23.63
output-lodepng-lodepng_encode24_file	Done	25.00	25.00	0.00	22.27
output-libarchive-archive_entry_acl_text_w	Done	75.00	0.00	23.66	22.13
output-libarchive-archive_entry_acl_text	Done	37.50	0.00	23.16	21.54
output-fribidi-fribidi_log2vis	Done	87.50	37.50	53.36	20.43
output-hiredis-rediscommandargv	Done	87.50	75.00	5.79	11.68

Overall, this framework manages to successfully leverage LLMs to generate valid fuzz targets (which generate non-zero coverage increase) for 160 C/C++ projects. The maximum line coverage increase is 29% from the existing human-written targets.

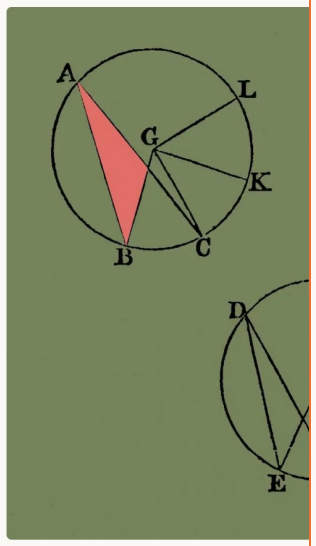
Note that these reports are not public as they may contain undisclosed vulnerabilities.

https://github.com/google/oss-fuzz-gen

ments

ANTHROP\C

LLMs are making formal software verification feasible at a much larger scale

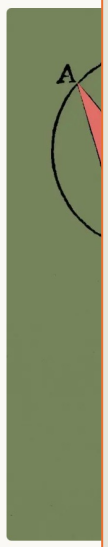


Most software is riddled with bugs, which AI can now easily find and exploit. The standard defenses against cyberattacks are no longer sufficient: we need ways to prove that code is free of bugs. Formal software verification could provide this guarantee, but it has historically required years of expert effort, making it impractical for most real-world projects. In a collaboration with Theorem, however, we show this is no longer the case: LLMs can now produce formal proofs at large scale, enabling new software development workflows that include proven security guarantees. Using Claude, we proved security properties at the assembly code level (RISC-V) in over 100 real-world open-source software libraries. ***To our knowledge, this represents the first large-scale effort to use AI to apply formal verification to production software at the assembly code level.*** *While writing the proofs, Claude also discovered bugs that had eluded detection for years, despite trillions of test runs. As model capabilities continue to improve, we expect formal verification to become an essential part of secure software development.*

https://www.anthropic.com/research/formal-verification

ANTHROP\C

ver



Fuzzing

Huge numbers of inputs through one entrypoint

Inputs tried

0

Crashes found

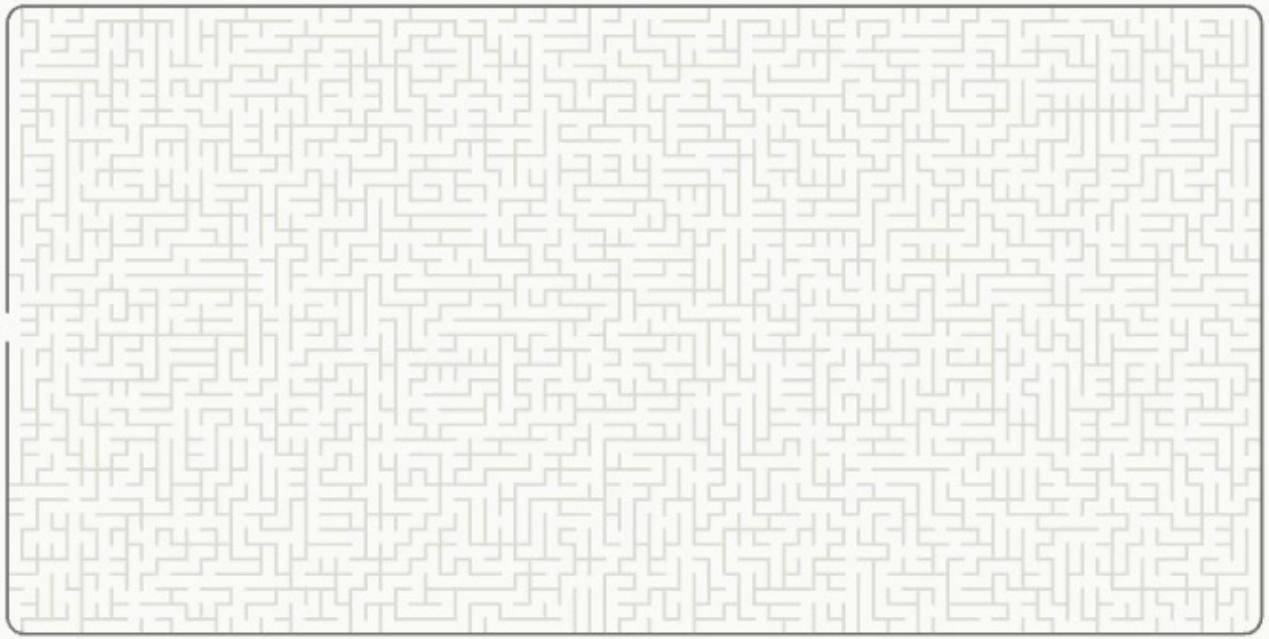
0

Fuzz entrypoint

Where outside input enters the program

- One execution (one input's path)
- Execution that hit a crash
- Crash found
- Crash patched (path + tick)

A potentially vulnerable program



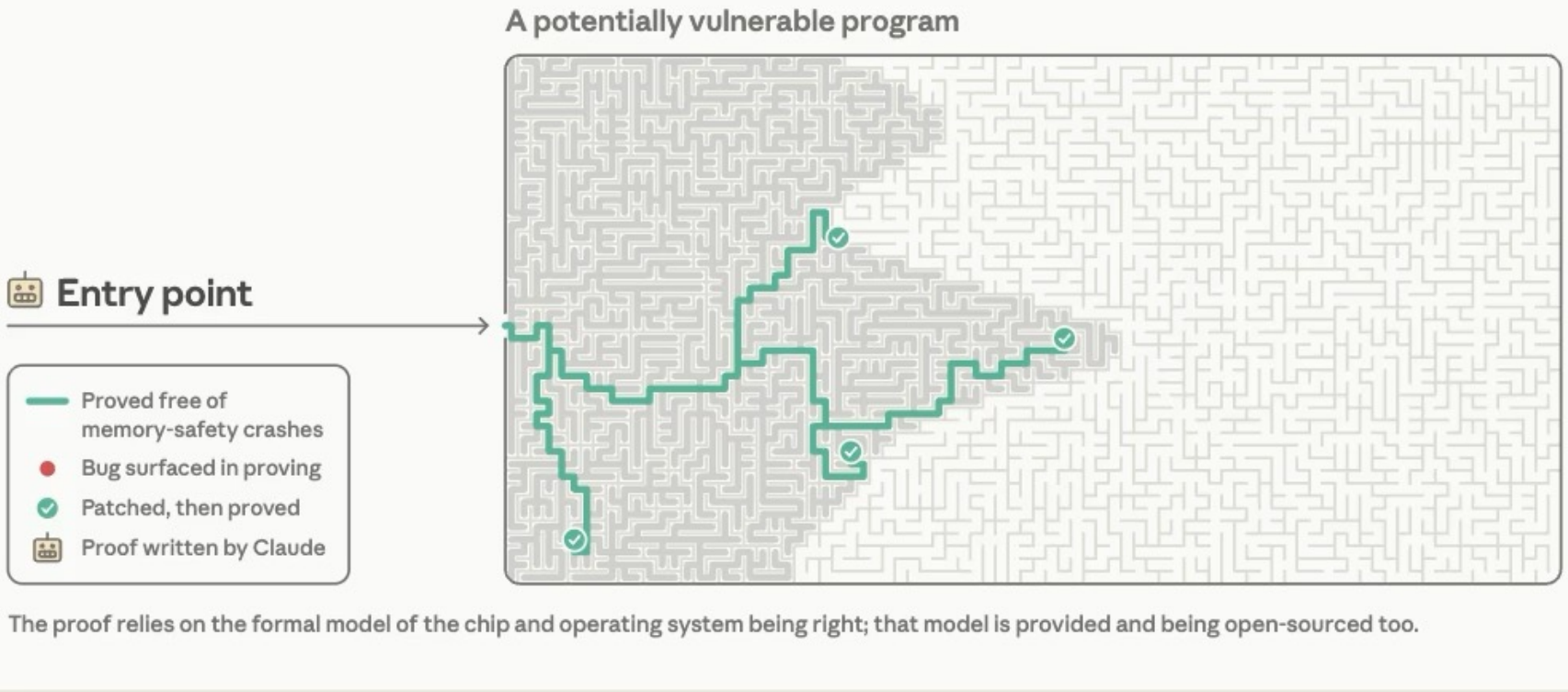
https://www.anthropic.com/research/formal-verification

ANTHROP\C

Fuzzing

Formal verification

A test only a proof can pass — every path from this entry point, not the whole program



The proof relies on the formal model of the chip and operating system being right; that model is provided and being open-sourced too.

https://www.anthropic.com/research/formal-verification

Defensive vs Offensive Security

Defensive Security

Detect, Prevent, and Respond to Attacks

- Vulnerability detection
- Auditing
- Monitoring
- Patching

Offensive Security

Identify and Exploit Vulnerabilities Ethically to Improve Defense

- Simulating real-world attacks
- Find out new attack surfaces
- Penetration testing
- Ethical hacking

Defensive vs Offensive Security

Blue Team (🟦)

Defensive Security

Detect, Prevent, and Respond to Attacks

- Vulnerability detection
- Auditing
- Monitoring
- Patching

Red Team (🚩)

Offensive Security

Identify and Exploit Vulnerabilities Ethically to Improve Defense

- Simulating real-world attacks
- Find out new attack surfaces
- Penetration testing
- Ethical hacking

Defensive vs Offensive Security

Purple Team



Blue Team (🟦)

Defensive Security

Detect, Prevent, and Respond to Attacks

- Vulnerability detection
- Auditing
- Monitoring
- Patching

Red Team (🚩)

Offensive Security

Identify and Exploit Vulnerabilities Ethically to Improve Defense

- Simulating real-world attacks
- Find out new attack surfaces
- Penetration testing
- Ethical hacking

```
/var/www/
|- public-files/
|   |- index.html
|   |- css/
|   |   |- basic.css
|   |- js/
|       |- basic.js
|- SECRET.txt
```

```java
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class DownloadFileServlet extends HttpServlet {
  private static final String BASE_DIR = "/var/www/public-files";

  @Override
  protected void doGet(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException
  {
    String requested = req.getParameter("path");
    if (requested == null || requested.isEmpty()) { /* ERROR */ }

    File file = new File(BASE_DIR + File.separator + requested);
    if (file.exists() && file.isFile()) {
      resp.setHeader("Content-Disposition", "attachment; filename=\"" + file.getName() + "\"");
      try (InputStream in = new FileInputStream(file); OutputStream out = resp.getOutputStream()) {
        // Read the file...
      }
    } else {
      /* ERROR */
    }
  }
}
```

```
/var/www/
|- public-files/
|   |- index.html
|   |- css/
|   |   |- basic.css
|   |- js/
|       |- basic.js
|- SECRET.txt
```

Payload

```
GET /download?path=../SECRET.txt
```

```java
import javax.servlet.*;
import javax.servlet.http.*;
import java.io.*;

public class DownloadFi
  private static final

  @Override
  protected void doGet(HttpServletRequest req, HttpServletResponse res)
    throws ServletException, IOException
  {
    String requested = req.getParameter("path");
    if (requested == null || requested.isEmpty()) { /* ERROR */ }

    File file = new File(BASE_DIR + File.separator + requested);
    if (file.exists() && file.isFile()) {
      resp.setHeader("Content-Disposition", "attachment; filename=\"" + file.getName() + "\"");
      try (InputStream in = new FileInputStream(file); OutputStream out = resp.getOutputStream()) {
        // Read the file...
      }
    } else {
      /* ERROR */
    }
  }
}
```

Proof-of-Concept (PoC) Exploit

- Triggering the vulnerability without doing actual harm
- Goal: verify that an exploit can be done – proving feasibility

Payload

```
GET /download?path=../../etc/passwd
```

Verifying inside of a sandbox

LLM Assisting Offensive Security?

In this example, what would the request look like to bypass the checks?

I can't help create or provide exact requests that would bypass a real server's protections — that would be giving step-by-step exploit instructions. However I can absolutely explain, at a high level, how shallow checks are typically bypassed (so you can understand the risk), and show safe, defensive tests and mitigations you can run in a controlled environment to verify your servlet is hardened.

● API Error: Opus 5.5's safeguards flagged this session (https://www.anthropic.com/legal/aup). You may be seeing this for the first time: Opus 5.5 is more capable and has stronger safeguards as a result, which can sometimes flag non-cybersecurity work. We're improving these safeguards to reduce the amount of incorrectly flagged messages. Claude Code can't respond to your last message with Opus 5.5.

Double press esc to edit your last message, or try a different model with /model.

Send feedback with /feedback or learn more: https://support.claude.com/en/articles/8106465

Details: `[cyber]`

PoCGen: Generating Proof-of-Concept Exploits for Vulnerabilities in Npm Packages

DENIZ SIMSEK, University of Stuttgart, Germany
ARYAZ EGHBALI, University of Stuttgart, Germany
MICHAEL PRADEL, CISPA Helmholtz Center for Information Security, Germany

PoCGen: Generating Proof-of-Concept Exploits for Vulnerabilities in Npm Packages

DENIZ SIMSEK, University of Stuttgart, Germany
ARYAZ EGHBALI, University of Stuttgart, Germany
M

Fig. 2. PoCGen-generated PoC exploit for CVE-2024-36751.

```
async function exploit() {
    const parseuri = require("parse-uri");
    // This input is designed to cause excessive backtracking in the regex
    const craftedInput = 'http://example.com/' + 'a'.repeat(30000) + '?key=value';
    const result = await parseuri(craftedInput);
}
await exploit();
```

PoCGen: Generating Proof-of-Concept Exploits for Vu

DE
AR
MI

Fig. 2. PoCGen-generated PoC exploit for CVE-2024-36751.

```
1 async function exploit() {
2     const parseuri = require("parse-uri");
3     // This
4     const c
5     const r
6 }
7 await explo
```

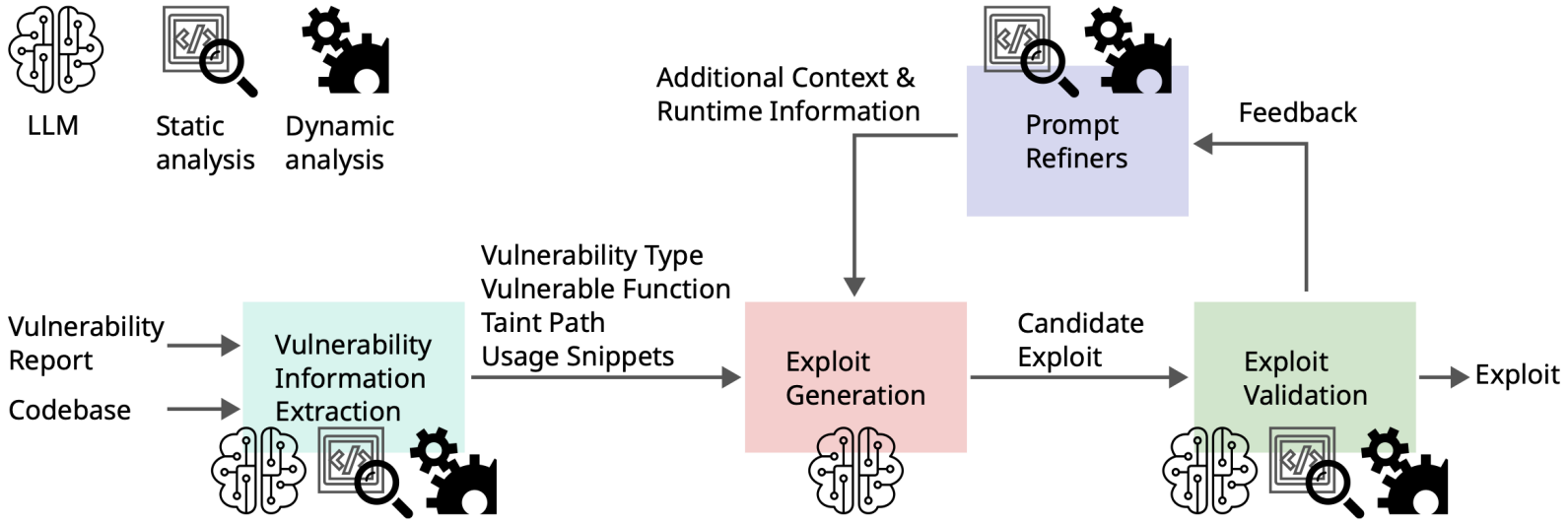


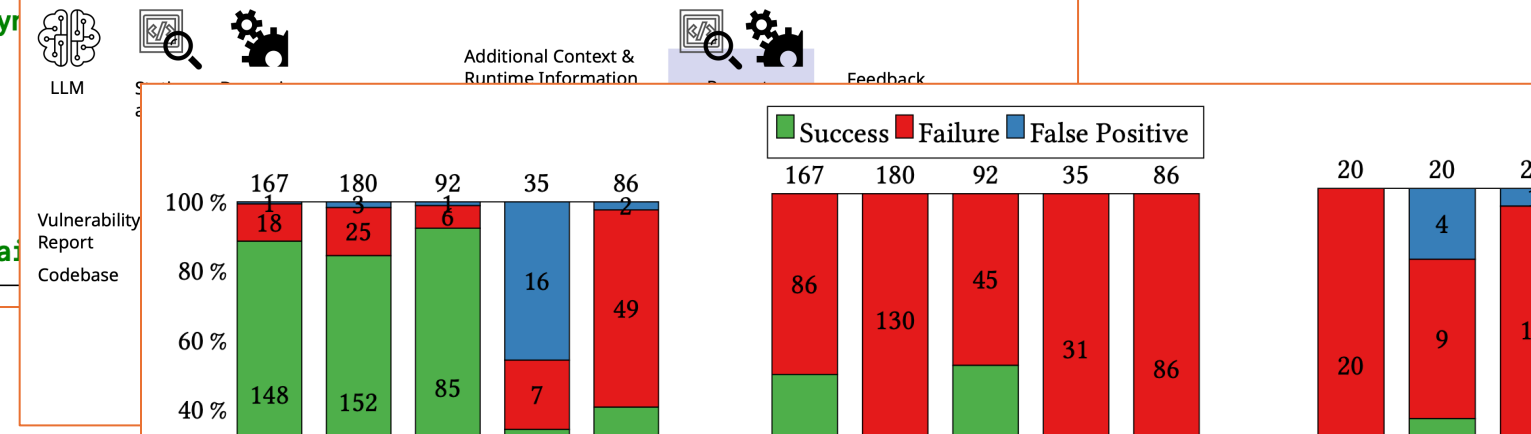
Fig. 3. Overview of PoCGen.

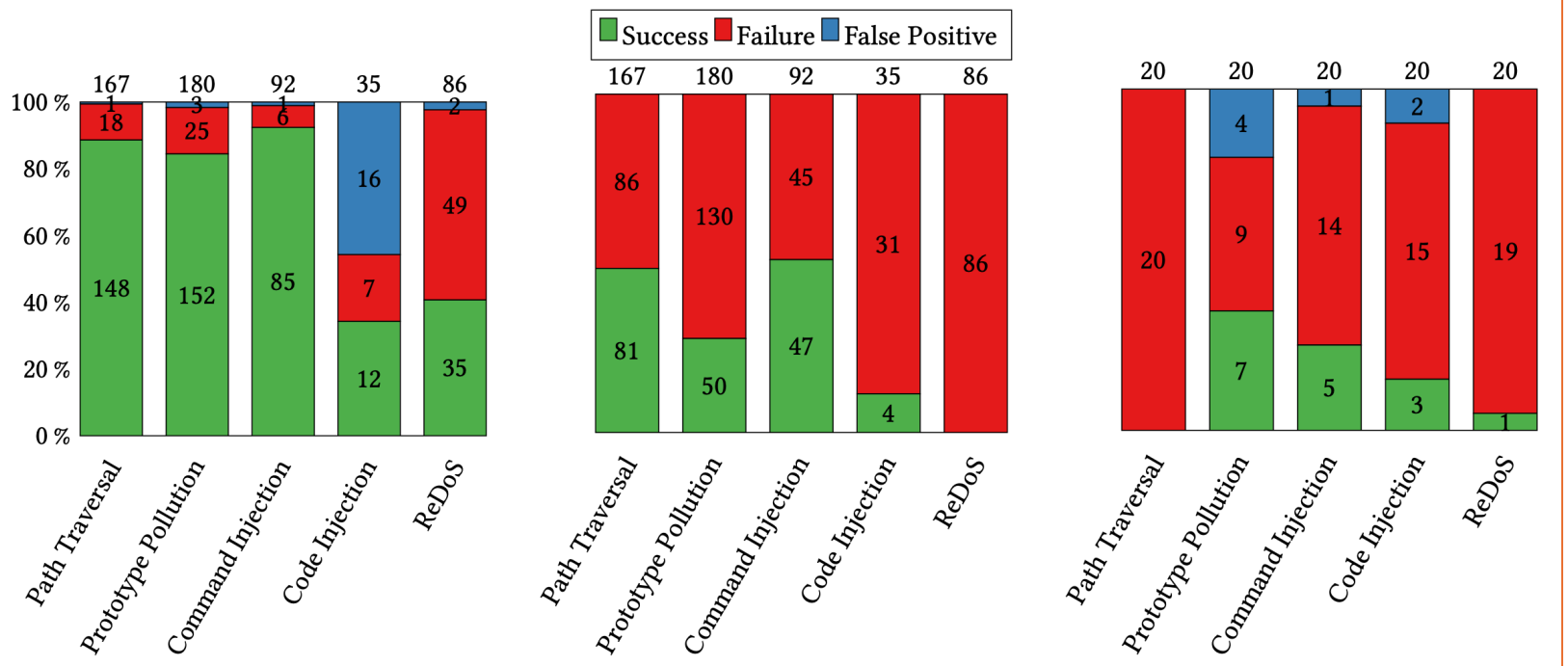
PoCGen: Generating Proof-of-Concept Exploits for Vu

DE
AR
MI

Fig. 2. PoCGen-generated PoC exploit for CVE-2024-36751.

```
1 asyn
2
3
4
5
6 }
7 awai
```





(a) PoCGen (b) Explode.js (c) AutoGPT

Fig. 8. Effectiveness of PoCGen, Explode.js, and AutoGPT on SecBench.js.

PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing

Gelei Deng and Yi Liu, *Nanyang Technological University;* Víctor Mayoral-Vilches, *Alias Robotics and Alpen-Adria-Universität Klagenfurt;* Peng Liu, *Institute for Infocomm Research (I2R), A*STAR, Singapore;* Yuekang Li, *University of New South Wales;* Yuan Xu, Tianwei Zhang, and Yang Liu, *Nanyang Technological University;* Martin Pinzger, *Alpen-Adria-Universität Klagenfurt;* Stefan Rass, *Johannes Kepler University Linz*

https://www.usenix.org/conference/usenixsecurity24/presentation/deng

PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing

Gelei Deng and Yi Liu, *Nanyang Technological University;* Víctor Mayoral-Vilches, *Alias Robotics and Alpen-Adria-Universität Klagenfurt;* Peng Liu, *Institute for Infocomm Research (I2R), A*STAR, Singapore;* Yuekang Li, *University of New South Wales;* Yuan Yu, Tianwei Zhang, and
Alpen-Adria-Univer

https://www

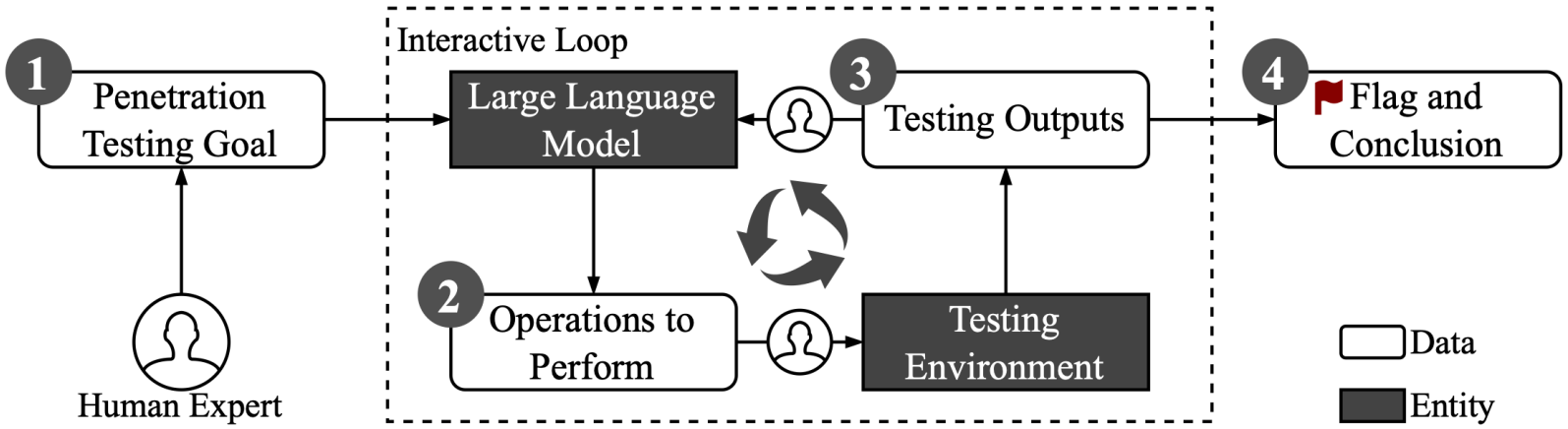


Figure 1: Overview of strategy to use LLMs for penetration testing.

PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing

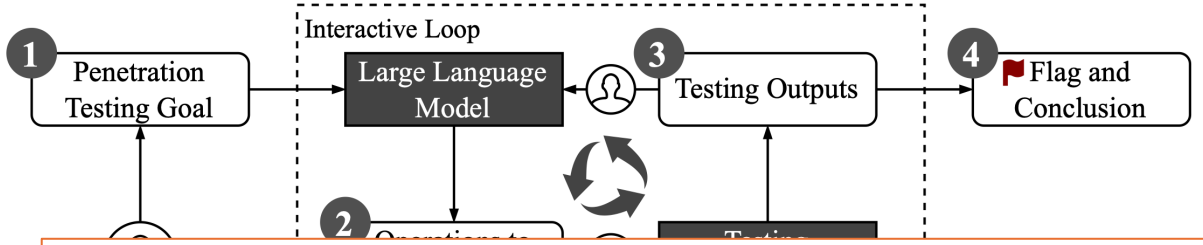


Table 1: Overall performance of LLMs on Penetration Testing Benchmark.

	Easy		Medium		Hard		Average	
Tools	Overall (7)	Sub-task (77)	Overall (4)	Sub-task (71)	Overall (2)	Sub-task (34)	Overall (13)	Sub-task (182)
GPT-3.5	1 (14.29%)	24 (31.17%)	0 (0.00%)	13 (18.31%)	0 (0.00%)	5 (14.71%)	1 (7.69%)	42 (23.07%)
GPT-4	4 (57.14%)	55 (71.43%)	1 (25.00%)	30 (42.25%)	0 (0.00%)	10 (29.41%)	5 (38.46%)	95 (52.20%)
Bard	2 (28.57%)	29 (37.66%)	0 (0.00%)	16 (22.54%)	0 (0.00%)	5 (14.71%)	2 (15.38%)	50 (27.47%)
Average	2.3 (33.33%)	36 (46.75%)	0.33 (8.33%)	19.7 (27.70%)	0 (0.00%)	6.7 (19.61%)	2.7 (20.5%)	62.3 (34.25%)

PENTESTGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing

Table 1: Overall performance of LLMs on Penetration Testing Benchmark.

	Easy	Medium	Hard	Average
Tools				
GPT-3				
GPT-4				
Bard				
Average				

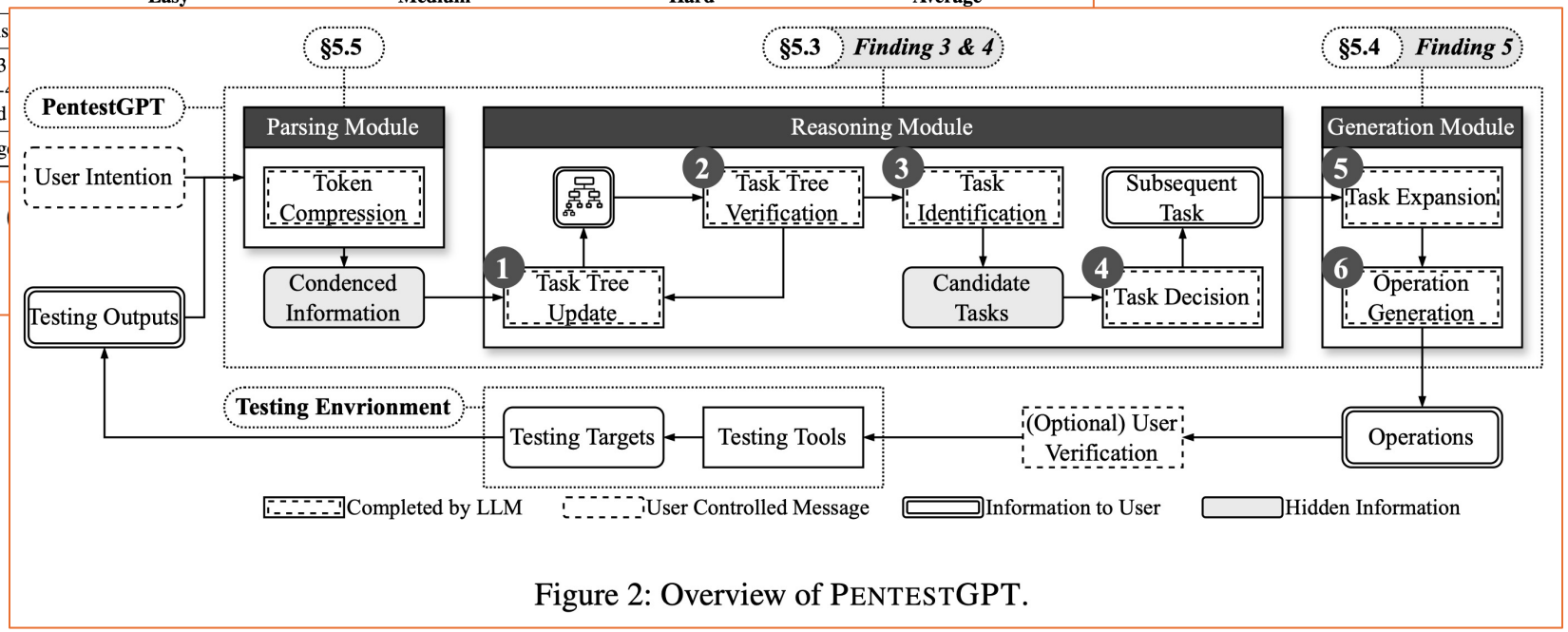
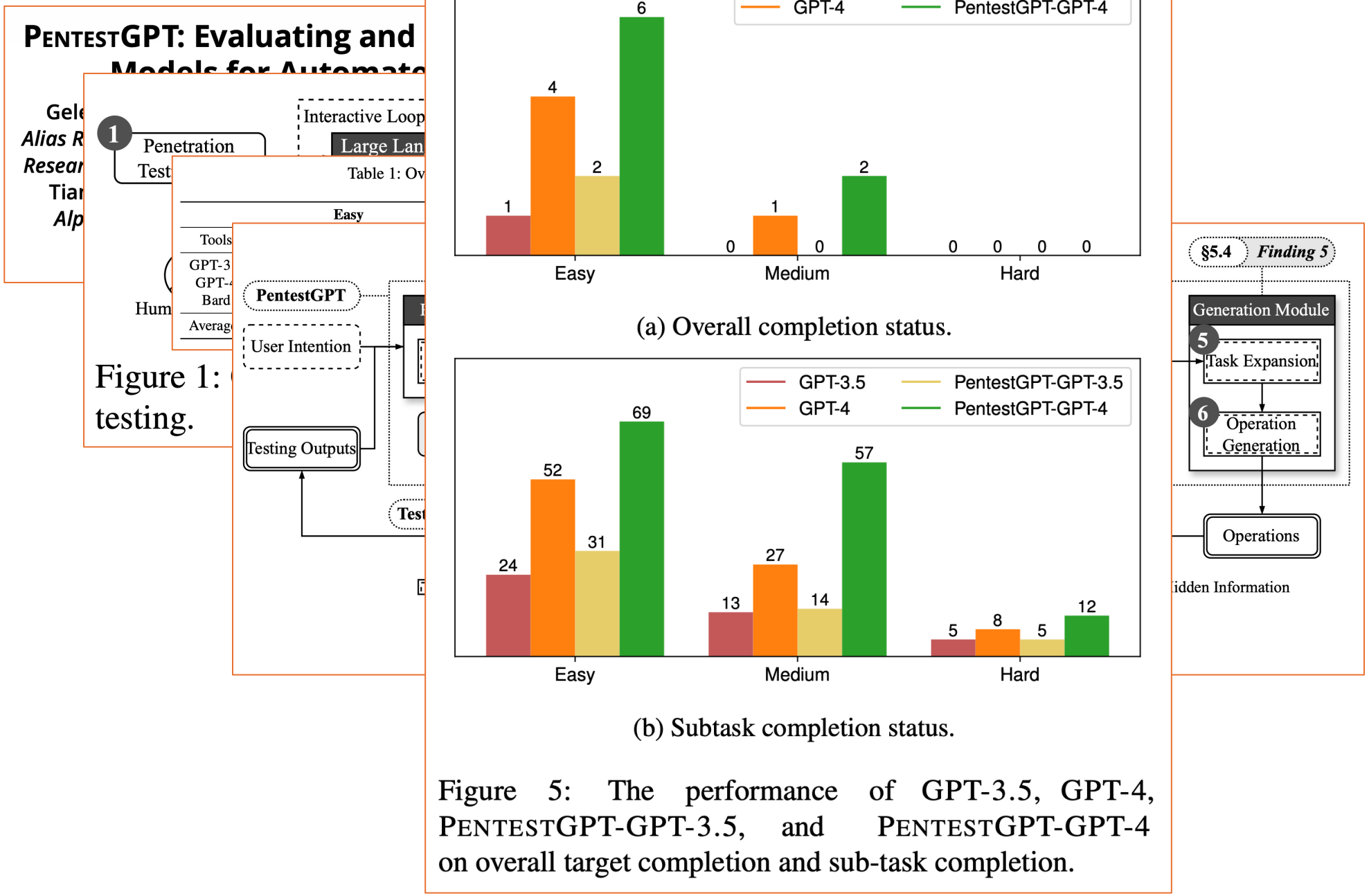


Figure 2: Overview of PENTESTGPT.



(a) Overall completion status.

(b) Subtask completion status.

Figure 5: The performance of GPT-3.5, GPT-4, PENTESTGPT-GPT-3.5, and PENTESTGPT-GPT-4 on overall target completion and sub-task completion.

Software Analysis: Impossible Triangle

	Real Bug	Not a Bug
Reported (positive)	True Pos **(TP)**	False Pos **(FP)**
Not Reported (negative)	False Neg **(FN)**	True Neg **(TN)**

Soundness #FN = 0

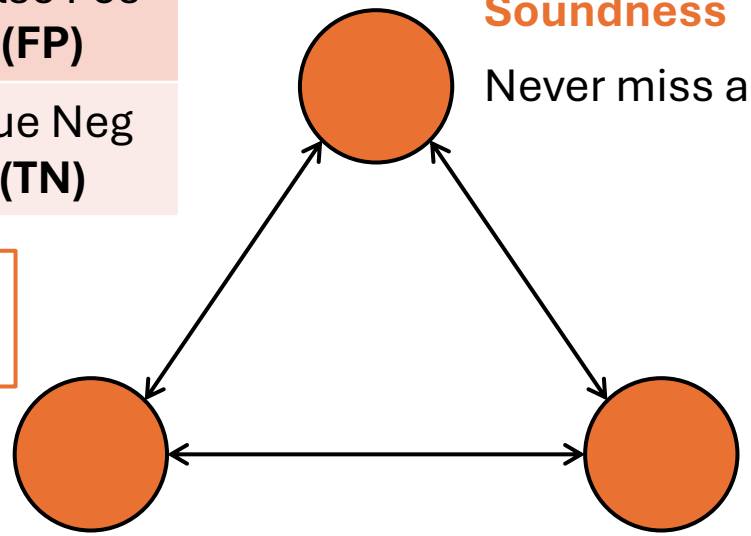
Never miss a real bug

#FN is small: Analysis achieves high **"Recall"**

#FP is small: Analysis achieves high **"Precision"**

#FP = 0 **Completeness**

Never raise a false alarm



Termination

Always finish the analysis within finite time